UNIVERSITÀ DI PARMA

Dipartimento di Ingegneria e Architettura

Corso di Laurea in Ingegneria Informatica Magistrale

SegnalER: progettazione e sviluppo di un motore di ricerca semantico per segnalazioni sanitarie

SegnalER: design and development of a semantic search engine for incident reports in health care

Relatore:

Prof. Stefano Cagnoni

Correlatore:

Dott. Vieri Emiliani

Tesi di Laurea di:

Alessandro Muzzi

ANNO ACCADEMICO 2020-2021

Ringraziamenti

Questa tesi segna il compimento dei miei anni di studio, sono fiero del mio percorso e se tornassi indietro rifarei tutto allo stesso modo (ma facciamo che ho finito e non si torna indietro).

Ringrazio il mio relatore Stefano Cagnoni per avermi seguito in questa laurea così come nella triennale.

Ringrazio il mio correlatore, Vieri Emiliani, per la magistrale guida durante tutto il percorso di tirocinio, senza il quale questo progetto non sarebbe sicuramente riuscito altrettanto bene.

Ringrazio la mia famiglia per avermi sempre supportato e creduto in me durante questo percorso tutt'altro che facile ma ricco di immense soddisfazioni.

Un grazie importante va ad Alessia, una persona molto speciale per me, che sa sempre come starmi a fianco e come supportarmi, vi auguro sinceramente di trovare una persona che sia per voi, ciò che lei è per me.

Infine, un grazie a tutti i miei parenti, amici e conoscenti che mi vogliono bene e che mi hanno sostenuto e motivato durante questo percorso, non farò altri nomi per non indispettire nessuno, ma chi mi è stato accanto sa che questi ringraziamenti sono anche per lui.

Indice

1 Introduzione **1**

1.1 Struttura della tesi 2

2 Stato dell'arte **3**

2.1 Bag Of Words (BOW) 4

2.2 Embedding 6

2.2.1 Word2Vec 7

2.2.1.1 CBOW 8

2.2.1.2 Skip gram 8

2.2.2 GloVe: Global Vector for word representation 9

2.2.3 FastText 9

2.2.4 Limiti dei word embedding 11

2.3 Sentence embedding 12

2.3.1 Average 13

2.3.2 IDF 13

2.3.3 SIF 14

2.3.4 Transformer 15

2.3.4.1 Encoder e decoder 16

2.3.4.2 Attention 17

2.3.4.3 BERT 18

2.3.4.4 sBERT 19

2.4 Document embedding 21

2.5 Metriche di confronto 22

2.5.1 Word mover's distance 22
2.5.2 Euclidean distance 23
2.5.3 Cosine similarity 24
2.5.4 Ranking 25
2.5.4.1 Explicit Relevance Feedback 25
2.5.4.2 Implicit Relevance Feedback 27
2.5.4.3 Pseudo Relevance Feedback 27

3 Architettura 29
3.1 API 30
3.1.1 GraphQL 31
3.2 Business Logic 36
3.3 Embedding Service 37
3.4 Elasticsearch 37
3.4.1 Elasticsearch: indici 38
3.4.2 Elasticsearch: ricerca 39
3.4.2.1 Query di ricerca 40
3.5 Integrazione in SegnalER 42
3.5.1 Velocizzazione delle ricerche con Approximated Nearest Neighbor 49

4 Fase di studio 53
4.1 Dataset e problematiche 54
4.1.1 Dati sporchi 54
4.1.2 Scarsità di dati 55
4.1.3 Interpretazione degli embedding 55
4.2 Sviluppo e confronto modelli 57
4.2.1 Word embedding 57
4.2.2 Sentence embedding 62

5 Conclusioni 69
5.1 Sviluppi futuri 71

Bibliografia **73**

Capitolo 1

Introduzione

L'obiettivo della tesi è lo sviluppo di un motore di ricerca semantico per segnalazioni in ambito sanitario, ovvero un motore di ricerca in grado di fornire risultati che non necessariamente contengano le parole ricercate ma che siano affini a queste nel significato.

Il progetto SegnalER nasce dall'esigenza della regione Emilia Romagna di avere una piattaforma unificata per raccogliere, catalogare e consultare le segnalazioni sanitarie delle proprie AUSL. Con la sua offerta innovativa e altamente competitiva, Maps Group si è aggiudicata il bando regionale proponendo una soluzione comprensiva di diverse componenti AI, tra cui

- **il processo di importazione delle segnalazioni**, in grado di elaborare audio e testi non strutturati per generare segnalazioni gestibili dal sistema
- **un motore di correlazione**, necessario per evitare l'introduzione di segnalazioni duplicate e per fornire suggerimenti sulla categorizzazione delle stesse
- **un motore di ricerca semantico**, che sia in grado di effettuare ricerche in modo rapido e intelligente, sfruttando la semantica per ottenere i risultati più attinenti a quanto richiesto dall'utente

In questa tesi approfondiremo appunto l'ultima di queste componenti, il motore di ricerca semantico.

1.1 Struttura della tesi

Dopo l'introduzione, il lavoro di tesi continua con il capitolo 2 dove viene trattato il corrente stato dell'arte in ambito di ricerca semantica. Successivamente, il Capitolo 3 descrive l'architettura del motore di ricerca e la sua integrazione in SegnalER. Il Capitolo 4 riguarda le problematiche affrontate e la valutazione attraverso esperimenti dei diversi approcci sperimentati durante la realizzazione del progetto. Occorre infatti effettuare le giuste scelte in relazione al contesto in cui ci si trova, alle richieste fatte dal cliente e ai dati che egli ha fornito e che fornirà quando il prodotto sarà in produzione; queste decisioni sono quindi determinanti per garantire un corretto sviluppo della soluzione e di rispettare quanto stabilito nella proposta fatta al committente. L'ultimo capitolo, raccoglie le conclusioni, le impressioni personali e gli eventuali sviluppi futuri inerenti a questo progetto di tesi.

Capitolo 2

Stato dell'arte

Gli esseri umani possono spesso decidere senza sforzo la somiglianza o la relazione fra due parole. Ciò può essere spiegato, in parte, dall'esperienza che gli esseri umani hanno nell'usare e incontrare parole correlate in contesti simili. Ad esempio, come esseri umani, sappiamo che *pioggia* e *ombrello* sono strettamente correlati, mentre c'è una piccola connessione, se non nessuna, tra *pioggia* e *libro di testo*. Sebbene ciò sia banale per gli esseri umani, spesso non è così semplice tradurre questo processo di giudizio per le macchine senza l'attenta formulazione del background e della conoscenza contestuale che circonda ogni parola e le sue relazioni. Formalmente, la relazione semantica (SR) è definita come una forma di associazione semantica o funzionale tra due parole , che va al di là delle più semplici relazioni lessicali come sinonimia e iponimia[16]. L'obiettivo dei metodi SR è modellare tali associazioni. La SR è ampiamente utilizzata in molte applicazioni pratiche, in particolare nell'elaborazione del linguaggio naturale (NLP), nell'ambito delle quali si segnalano il recupero delle informazioni semantiche, l'estrazione di parole chiave e il riepilogo dei documenti, all'interno dei quali viene utilizzata per quantificare le relazioni tra parole o tra parole e documenti[17]. Proprio utilizzando tecniche appartenenti a quest'ultimo sottoinsieme, i motori di ricerca semantici riescono a fornire risultati analoghi a quelli desiderati in questo progetto.

2.1 Bag Of Words (BOW)

Il Bag Of Words è una tecnica semplice e popolare per l'estrazione di features dal testo. Il modello BOW elabora il testo per trovare quante volte ciascuna parola è apparsa nella frase, con un procedimento detto di vettorizzazione:

1. Separare il testo in frasi
2. Separare le frasi in parole
3. Rimuovere punteggiatura e *stop-words*[1]
4. Convertire tutte le parole in minuscolo
5. Calcolare la distribuzione statistica delle parole all'interno delle frasi

	it	is	puppy	cat	pen	a	this
it is a puppy	1	1	1	0	0	1	0
it is a kitten	1	1	0	0	0	1	0
it is a cat	1	1	0	1	0	1	0
that is a dog and this is a pen	0	2	0	0	1	2	1
it is a matrix	1	1	0	0	0	1	0

Figura 2.1: Esempio di matrice sparsa ottenuta con BOW[15]

In questo modo è possibile confrontare la somiglianza di 2 frasi osservando come si collocano nello spazio del nostro vocabolario.

[1]Le stop words sono un elenco di parole ritenute non significative, che viene filtrato prima o dopo l'elaborazione dei dati del linguaggio naturale.

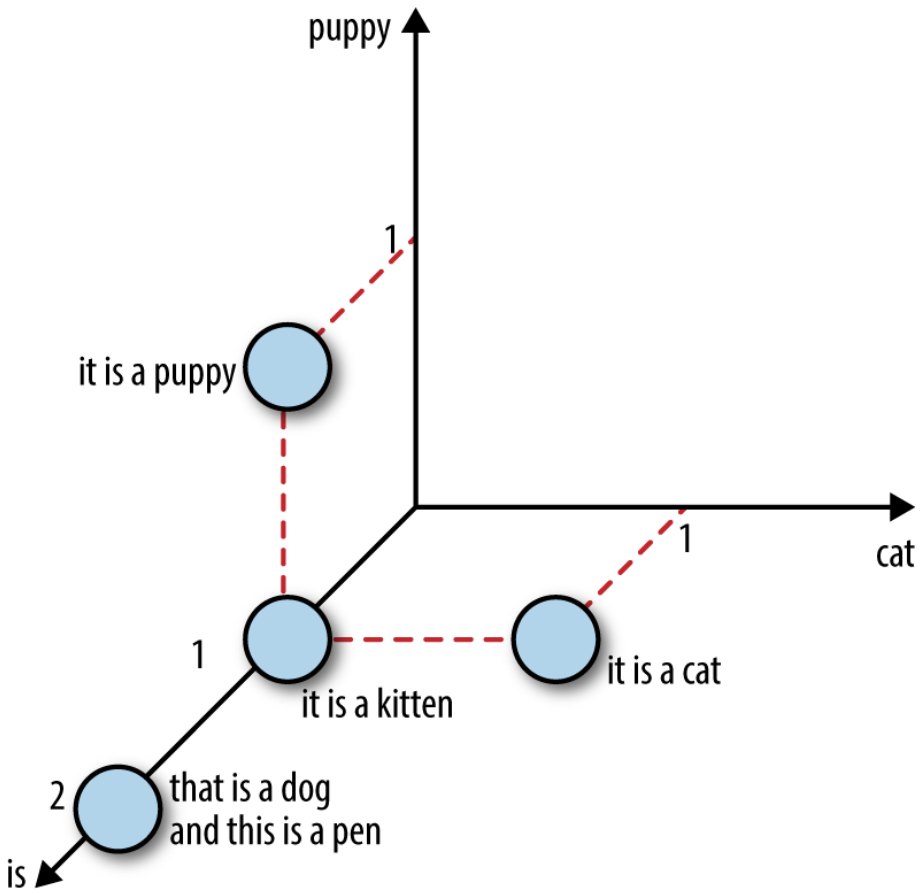


Figura 2.2: Posizionamento di alcune frasi nello spazio composto da *puppy*, *cat*, *is*[15]

Nell'esempio in figura 2.2, possiamo osservare come le frasi *"it is a puppy"*, *"it is a cat"*, *"it is a kitten"*, *"that is a dog and this is a pen"* si posizionino diversamente nello spazio vettoriale composto dalle parole *puppy*, *cat*, *is*. Ora, immaginando lo spazio *n-dimensionale* di un vocabolario, si riesce meglio a visualizzare la matrice sparsa di BOW, dove ogni frase ha il proprio vettore n-dimensionale rappresentante la propria posizione semantica all'interno del vocabolario. Si nota dunque subito come, all'aumentare di dimensione del vocabolario, questa matrice diventi sempre più *sparsa*, dal momento che una singola frase conterrà poche della totalità delle parole contenute in un vocabolario. Questa struttura porta ovviamente a problemi di scalabilità e di difficoltà nel computare matrici di dimensioni così elevate come quelle che

servirebbero in ambito di machine learning, oltre a non rappresentare tutte le caratteristiche di una frase, come ad esempio l'ordine delle parole. Per ovviare a questi problemi si è passati alle rappresentazioni *dense*, ovvero gli *embedding*.

2.2 Embedding

L'embedding di parole è una tecnica in cui le singole parole di un dominio o di una lingua sono rappresentate come vettori di valori reale in uno spazio dimensionale inferiore.

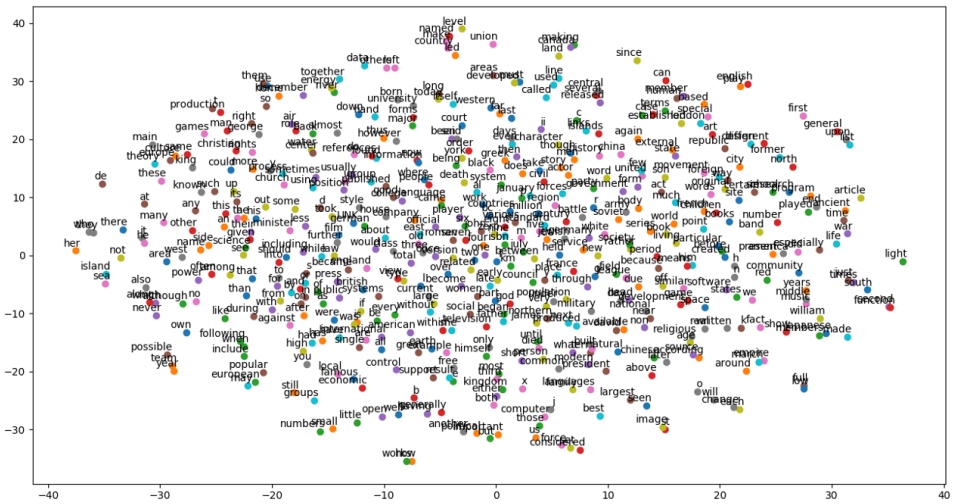


Figura 2.3: Embedding rappresentati in uno spazio 2D[18]

Gli embedding riescono a ridurre le dimensioni traducendo grandi vettori sparsi in uno spazio di dimensioni inferiori che preserva le relazioni semantiche. In questo modo il problema della matrice sparsa con BOW viene risolto mappando i dati in uno spazio a dimensione inferiore, così come il problema della mancanza di una relazione significativa viene risolto posizionando vettori di elementi semanticamente simili uno vicino all'altro, le parole che hanno un significato simile avranno quindi distanze minori nello spazio vettoriale. Come esempio pratico, osserviamo il ragionamento "il re sta alla regina come

l'uomo sta alla donna" sia codificato nello spazio vettoriale, consentendo di effettuare operazioni matematiche fra vettori sottraendone componenti per ottenere significati differenti:

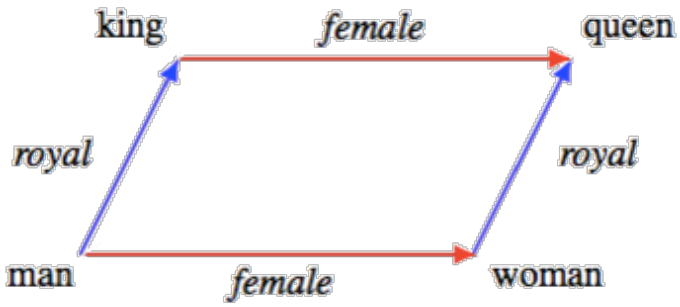


Figura 2.4: king - man + female = queen

Ma come riuscire effettivamente a ridurre le dimensioni?
Il modo che probabilmente verrà in mente per primo per ridurre le dimensioni di un vettore sparso è quello di utilizzare la Principal Component Analysis (PCA), la quale cerca di trovare le dimensioni più strettamente correlate in modo da poterle ridurre perdendo la minor quantità possibile di significato. In realtà, sebbene questa sia una strada percorribile, al giorno d'oggi si utilizzano tecniche di costruzione degli embedding che non prevedono il passaggio da una sparse matrix, eliminando così la necessità di grosse capacità di calcolo anche per la fase di costruzione degli embedding.

2.2.1 Word2Vec

Word2Vec è un algoritmo inventato da Google per l'addestramento degli embedding di parole. Word2Vec si basa sull'ipotesi distributiva, che afferma che *le parole che hanno spesso le stesse parole vicine, tendono ad essere semanticamente simili.* Questo aiuta a mappare parole semanticamente simili in embedding geometricamente vicini. L'ipotesi distributiva può utilizzare *CBOW* o *skip gram.*

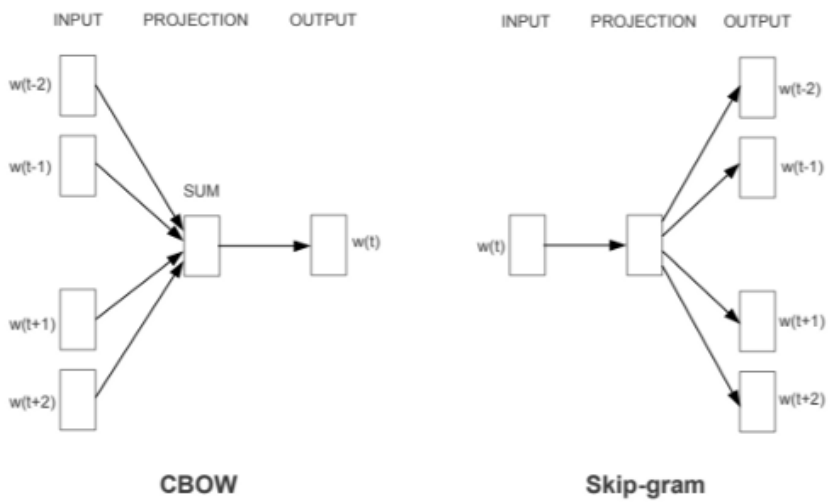


Figura 2.5: CBOW vs skip gram

2.2.1.1 CBOW

I modelli Word2Vec sono reti neurali poco profonde con uno strato di input, uno strato di proiezione e uno strato di output. Sono addestrati a ricostruire i contesti linguistici delle parole. Il livello di input per la rete neurale di Word2vec richiede un corpus di testo abbastanza ampio da poter produrre uno spazio vettoriale, in genere di diverse centinaia di dimensioni. Ad ogni parola univoca nel corpus del testo viene assegnato un vettore corrispondente nello spazio. Questa architettura è chiamata *Continuous Bag Of Words* (CBOW) in quanto utilizza la rappresentazione distribuita continua del contesto, considerando l'ordine delle parole sia prima che dopo la parola interessata. Ciò consente di posizionare i vettori di parole di contesti comuni nel corpus uno vicino all'altro nello spazio vettoriale.

2.2.1.2 Skip gram

Skip gram, al contrario, non prevede la parola corrente in base al contesto, ma utilizza ogni parola corrente come input per un classificatore log-lineare

con livello di proiezione continua e prevede le parole entro un determinato intervallo prima e dopo la parola corrente.

2.2.2 GloVe: Global Vector for word representation

GloVe è un progetto open-source sviluppato presso l'Università di Stanford e rilasciato per la prima volta nel 2014. Il suo nome, Global Vector, deriva dal fatto che le statistiche del corpus globale vengono acquisite direttamente dal modello.
Per farlo sfrutta

- metodi di fattorizzazione a matrice globale, come l'analisi semantica latente (LSA), per generare rappresentazioni di parole a bassa dimensione
- metodi basati su finestre di contesto locale, come il modello skip-gram

LSA sfrutta in modo efficiente le informazioni statistiche ma non funziona bene sull'analogia delle parole, fornendo così una struttura dello spazio vettoriale non ottimale. Metodi come skip-gram funzionano meglio nel rappresentare le analogie, ma utilizzano poco le statistiche del corpus poiché non sono addestrati sui conteggi globali di co-occorrenza. Senza addentrarci troppo nello specifico del funzionamento di GloVe, possiamo dire che utilizza un particolare modello in grado di rappresentare efficacemente le analogie e nel contempo di tener conto delle statistiche globali del corpus.

Nonostante GloVe riesca a performare meglio di tutti i metodi trattati fin'ora, c'è un ostacolo che nemmeno questo modello riesce a superare e che è invece fondamentale per la ricerca semantica: la gestione delle parole *Out Of Vocabulary* (OOV).

2.2.3 FastText

Il modello FastText, sviluppato da Facebook, è un metodo per apprendere la rappresentazione delle parole che si basa sul modello skip gram di Word2Vec

e ne migliora l'efficienza e le prestazioni:

- È più veloce e più semplice da addestrare. Sulla valutazione della somiglianza, FastText fornisce risultati migliori rispetto a Word2Vec su un corpus di addestramento più ridotto.

- Beneficia delle informazioni estratte dalle sottoparole (o *subwords*). Quando viene fornita una parola, FastText utilizza tutti gli n-grammi di questa parola per calcolare il punteggio della parola. Ad esempio, con la parola "artificiale", gli n-grammi con $n = 3$ sono
"ar, art, rti, tif, ifi, fic, ici, ial, ale, le".

- Grazie a questa gestione delle parole, FastText può partire dagli n-grammi per generare nuove parole Out Of Vocabulary. Il vettore di una parola OOV può essere costruito attraverso la rappresentazione vettoriale media dei suoi n-grammi.

Mentre con skip-gram la probabilità del contesto data una determinata parola t è parametrizzata dagli *word embedding*[2] attraverso una funzione di valutazione s:

$$s(w_t, w_c) = u_{w_t}^T v_{w_c}$$

dove u e v sono estratti rispettivamente dalle matrici di embedding di input ed output, con FastText la funzione di valutazione diventa:

$$s(w_t, w_c) = \sum_{g \in G_{w_t}} z_g^T v_{w_c}$$

con G_{w_t} set degli n-grammi della parola w_t e z_g come vettore del g-esimo n-gramma.

[2]Embedding relativi a parole, 1 parola $\leftrightarrow$ 1 embedding

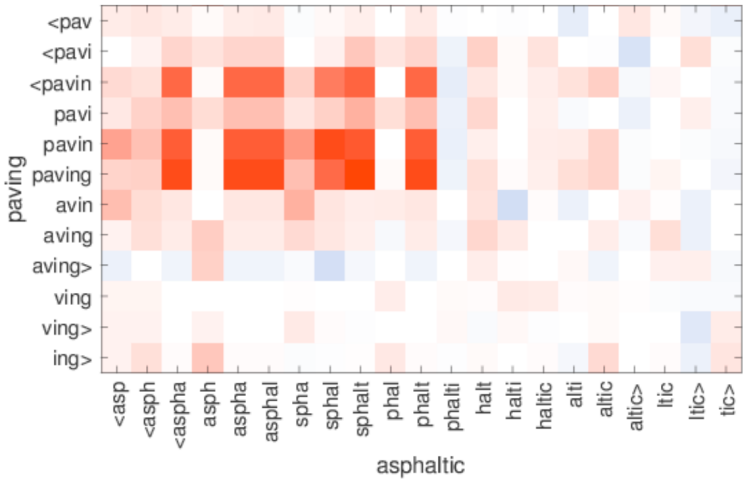


Figura 2.6: Visualizzazione della similarità tra gli n-grammi utilizzando parole OOV [1]

In figura 2.6 si può osservare un esempio di quanto appena discusso: sull'asse X vi è una parola OOV e le intersezioni tra gli n-grammi mostrano il loro grado di somiglianza tenendo conto che come criterio di valutazione viene utilizzata la cosine similarity tra i vettori degli n-grammi e che "rosso" e "blu" indicano rispettivamente un valore di similarità alto o uno basso.

L'utilizzo degli n-grammi per la generazione di parole OOV rende FastText la prima scelta tra i modelli proposti fino ad ora, offrendo tolleranza ad errori di battitura e riuscendo a cogliere con i medesimi embedding le sfumature date da suffissi e prefissi.

2.2.4 Limiti dei word embedding

Sia GloVe che FastText, sono in grado di produrre buoni risultati quando il task è di similarità sulle singole parole, mentre mostrano diverse difficoltà quando viene richiesta similarità tra frasi. Il problema è che l'approccio non è sufficientemente accurato poiché non prende in considerazione l'ordine in cui le parole compaiono, il che porta alla perdita della comprensione sintattica e semantica della frase. Ad esempio, "Andrai lì per insegnare, non

per giocare" e "Andrai lì per giocare, non per insegnare" avranno la stessa rappresentazione nello spazio vettoriale ma non hanno lo stesso significato. Inoltre, il modello di embedding delle parole non può fornire risultati soddisfacenti su una grande quantità di dati di testo, poiché la stessa parola può avere un significato diverso in una frase diversa a seconda del contesto della stessa. Prendiamo, ad esempio, "Mi passeresti il *riso*?" e "Ho *riso* davvero tanto ieri sera". In entrambe le frasi, è presente la parola "*riso*" ma ha significati completamente diversi nelle due frasi: nella prima è un sostantivo, in particolare rappresenta un alimento, mentre nella seconda è un verbo. Abbiamo quindi bisogno di un tipo di rappresentazione che possa conservare il significato contestuale della parola presente in una frase. Un primo approccio per risolvere questo tipo di problematiche sono i *sentence embedding*; un altro sono gli embedding contestuali, approfonditi in Sezione 2.3.4.3.

2.3 Sentence embedding

I *sentence embedding* o *embedding di frasi* sono simili agli embedding descritti fin'ora ma, al posto della parola, viene codificata l'intera frase sotto forma di rappresentazione vettoriale. La rappresentazione vettoriale ottenuta conserva buone proprietà ereditando le proprie caratteristiche dagli embedding delle parole in essa contenute.

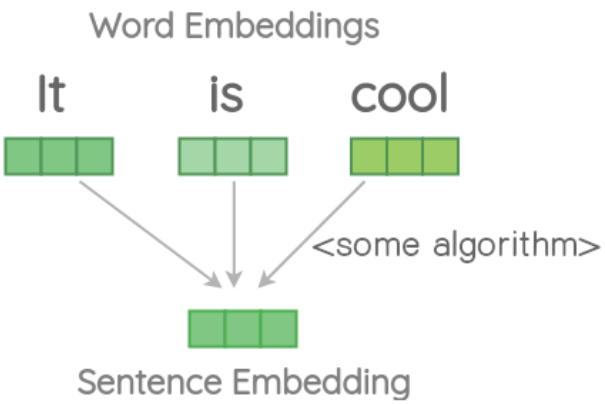


Figura 2.7: Word embedding → sentence embedding

Esistono diversi algoritmi per ottenere sentence embedding a partire da word embedding, tra questi approfondiremo

- Average
- IDF
- SIF

2.3.1 Average

É probabilmente il metodo più semplice e consiste nell'effettuare la media degli embedding di tutte le parole che compongono la frase.

2.3.2 IDF

Con IDF, *Inverse Document Frequency*, viene effettuata una media pesata degli embedding delle parole che compongono la frase, dove il peso è appunto l'IDF della parola in questione.
Definendo l'IDF come

$$idf(t, D) = log(\frac{N}{\mid d \in D : t \in d \mid})$$

con

- D: documenti componenti il corpus
- N: numero totale di documenti nel corpus, $N = D$
- $\mid d \in D : t \in d \mid$: numero di documenti in cui appare il termine t. Se il termine non dovesse apparire in alcun documento, ciò porterebbe ad una divisione per 0, è quindi buona norma aggiungere +1 al denominatore

risulta che i pesi saranno inversamente proporzionali alla loro frequenza nei diversi documenti; in questo modo vengono privilegiate le parole che riescono a discriminare meglio i documenti. Supponiamo, ad esempio, di avere un documento che parla di auto, uno di cibo e uno di aerei; è chiaro come articoli

o preposizioni non saranno utili a discriminare il contenuto in quanto sono parole contenute in tutti i documenti. Al contrario, valorizzare parole come "pneumatico", "carrozzeria", "mela", "ali" individua meglio il documento di appartenenza. Questo è esattamente quello che fa IDF nel regolare i pesi dei vari word embedding.

2.3.3 SIF

Anche SIF, *Smooth Inverse Frequency*, come IDF, ha come obiettivo concentrare l'attenzione degli embedding sulle parole meno frequenti e più discriminanti, ma il modo in cui lo fa è più raffinato ed elaborato della media ponderata vista in precedenza. Il paper di SIF, *A simple but tough-to-beat baseline for sentence embedding*[2], descrive l'algoritmo in pseudocodice:

Algoritmo 2.1 Sentence embedding

Input: Word embedding $\{v_w : w \in V\}$, a set of sentences S, parameter a and estimated probabilities $\{p(w) : w \in V\}$ of the words.
Output: Sentence embedding $\{v_s : s \in S\}$
  for all sentence s in S **do**
    $v_s \leftarrow \frac{1}{|s|} \sum_{w \in V} \frac{a}{a+p(w)} v_w$
  end for
  Form a matrix X whose columns are $\{vs : s \in S\}$, and let u be its first singular vector
  for all sentence s in S **do**
    $v_s \leftarrow v_s - uu^T v_s$
  end for

Senza entrare troppo nel merito del paper, l'algoritmo riceve in input gli word embedding delle parole presenti nel vocabolario V, un set di frasi S, un valore scalare a e una probabilità di occorrenza delle parole per ogni parola nel vocabolario e fornisce come output gli embedding delle frasi richieste. Per farlo, calcola il peso di ogni parola

$$\frac{a}{a + p(w)}$$

dove

- $p(w)$ è la probabilità di occorrenza della parola w che può essere ottenuta come $p(w) = \frac{\text{\# occorrenze di w}}{\text{\# tot di parole}}$

- a è un fattore di smorzamento, necessario per regolare i pesi delle parole e a far decrescere $\frac{a}{a+p(w)}$ al crescere di $p(w)$, è definito come $a = \frac{1-\alpha}{\alpha Z}$ dove α è un iperparametro e $Z \approx Z_{\tilde{c}_s}, \forall \tilde{c}_s$ con $Z_{\tilde{c}_s} = \sum_{w \in V} \exp(\langle \tilde{c}_s, v_w \rangle)$; l'approssimazione di Z è consentita dal momento che i v_w sono distribuiti uniformemente, di conseguenza $Z_{\tilde{c}_s}$ risulta circa uguale in qualsiasi direzione

dopodiché utilizza i pesi per calcolare gli embedding attraverso una media ponderata, normalizzandoli[3]

$$v_s \leftarrow \frac{1}{\mid s \mid} \sum_{w \in V} \frac{a}{a + p(w)} v_w$$

formando quindi una matrice X le cui colonne sono gli embedding v_s delle frasi ed estraendone il primo vettore singolare u. Infine, sottrae ad ogni sentence embedding la proiezione del vettore v_s su u, il suo componente principale.

$$v_s \leftarrow v_s - uu^T v_s$$

L'obiettivo è quello di non limitarsi a penalizzare le parole frequenti riducendo il loro peso nella composizione del sentence embedding, ma di rimuovere direttamente dall'embedding la parte che lo rende più simile a tutti gli altri.

2.3.4 Transformer

Il Transformer è un modello che evita la ricorrenza basandosi interamente su un meccanismo di attenzione per disegnare dipendenze globali tra input e output

---

Attention Is All You Need

Sebbene i sentence embedding riescano a colmare alcune lacune degli word embedding, non raggiungono di certo le prestazioni dei transformer, l'attuale

[3]Sebbene sia buona abitudine normalizzare gli embedding, non risulta necessario se utilizzati strettamente nel contesto della cosine similarity

stato dell'arte in ambito NLP. Il paper *Attention Is All You Need*[3] descrive i transformer ed il concetto di architettura *sequence-to-sequence*. Si tratta di una rete neurale che trasforma una data sequenza di elementi, come le parole in una frase, in una nuova sequenza; questi modelli sono infatti ottimi, ad esempio, nelle operazioni di traduzione del testo scritto. I trasformers si basano sul concetto di *Attention*, un meccanismo che osserva una sequenza in input e decide ad ogni step quale parte della sequenza è importante e quale è possibile scartare. A differenza di altri meccanismi sequence-to-sequence, i transformer si basano interamente sull'attention senza l'utilizzo di reti neurali ricorrenti (RNN). Le RNN erano infatti, fino ad ora, il miglior modo di catturare dipendenze temporali nelle sequenze ed i transformer hanno segnato un punto di svolta migliorandone notevolmente i risultati.

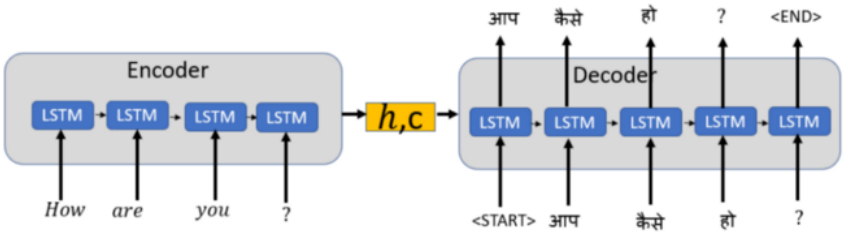


Figura 2.8: Rappresentazione semplificata di un transformer[19]

2.3.4.1 Encoder e decoder

Il concetto di encoder e decoder può essere paragonato ad un traduttore umano che conosce solo due lingue. La prima è la sua lingua madre mentre la seconda è un lingua intermedia che encoder e decoder condividono. Per tradurre, per esempio, il tedesco in francese, l'encoder converte l'input, in tedesco, nell'equivalente nella lingua intermedia che hanno in comune. L'output viene poi passato al decoder che lo traduce, infine, in francese. Il linguaggio intermedio condiviso fra encoder e decoder viene appreso da questi ultimi nella fase di training.

2.3.4.2 Attention

Il meccanismo di attenzione permette all'encoder e al decoder di "concentrarsi" solo su alcune parti dell'input. Ritornando all'esempio di Sezione 2.3.4.1, immaginiamo che l'encoder oltre a tradurre l'input nel linguaggio condiviso si "annoti" alcune parole chiave importanti per la semantica della frase e passi questa informazione aggiuntiva al decoder per facilitare la traduzione nel linguaggio in output. In altre parole, il meccanismo di attenzione prende in input due frasi, le converte in una matrice mettendo le parole della prima sequenza nelle righe e quelle della seconda nelle colonne. Mette poi in comune le parole identificando il contesto. Un esempio è visibile in Figura 2.9 dove le parole tradotte con la maggior confidenza sono evidenziate da celle di colore più acceso. Questo concetto può essere anche applicato a parole nella stessa frase stabilendo quanto esse sono correlate fra di loro, questo approccio prende il nome di *self-attention*. L'attention permette dunque di creare delle relazioni strette fra parole anche lontane nella sequenza ignorando quelle non correlate.

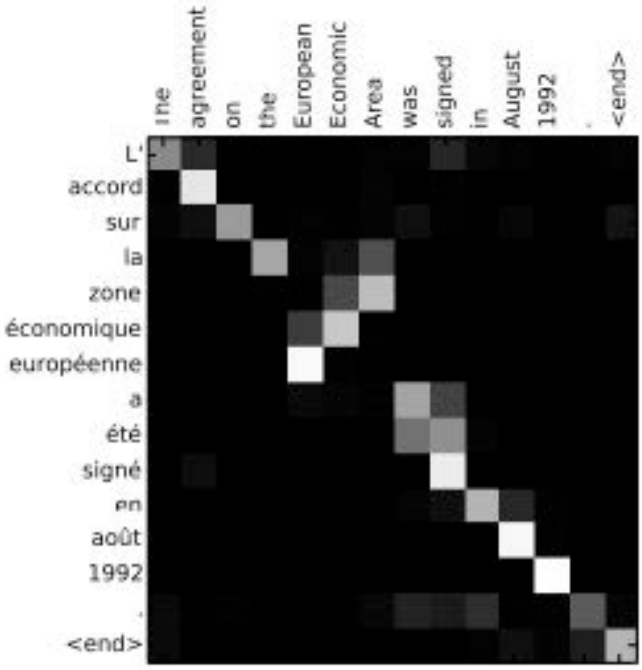


Figura 2.9: Esempio di matrice di attention

2.3.4.3 BERT

BERT, *Bidirectional Encoder Representations from Transformers*, fa riferimento ad un paper[4] pubblicato dai ricercatori di Google AI Language che ha rappresentato un punto di svolta importante ottenendo risultati rilevanti in vari task di NLP, n particolare nel campo del Question Answering e del Natural Language Inference. L'intuizione tecnica dietro BERT è quella di applicare l'addestramento bidirezionale dei Trasformers ai task di NLP. Ciò rappresenta una novità in quanto, fino ad ora, l'analisi del testo veniva eseguita principalmente leggendo le frasi da destra a sinistra o viceversa senza però osservarle contemporaneamente in entrambe le direzioni. Il paper mostra che un training bidirezionale permette di ottenere una comprensione più profonda del contesto e dello sviluppo della frase. BERT sfrutta, inoltre, una tecnica pensata per i task di computer vision: una rete neurale preaddestrata viene utilizzata come base di partenza per un processo di fine-tuning al fine di adattarla allo specifico problema in questione. Grazie a questo approccio, chiunque può fare finetuning di BERT per focalizzarlo sul proprio caso d'uso sfruttando il preaddestramento effettuato su un enorme dataset composto da *English Wikipedia*[4] e *BookCorpus*[10], un dataset composto da più di 10.000 libri appartenenti a diversi generi. Oltretutto, la disponibilità di due architetture di BERT, *Base* e *Large*, e del codice open-source[5], consente di modellare la propria soluzione basata su BERT in base alle proprie esigenze.

[4]https://en.wikipedia.org/wiki/English_Wikipedia
[5]https://ai.googleblog.com/2018/11/open-sourcing-bert-state-of-art-pre.html

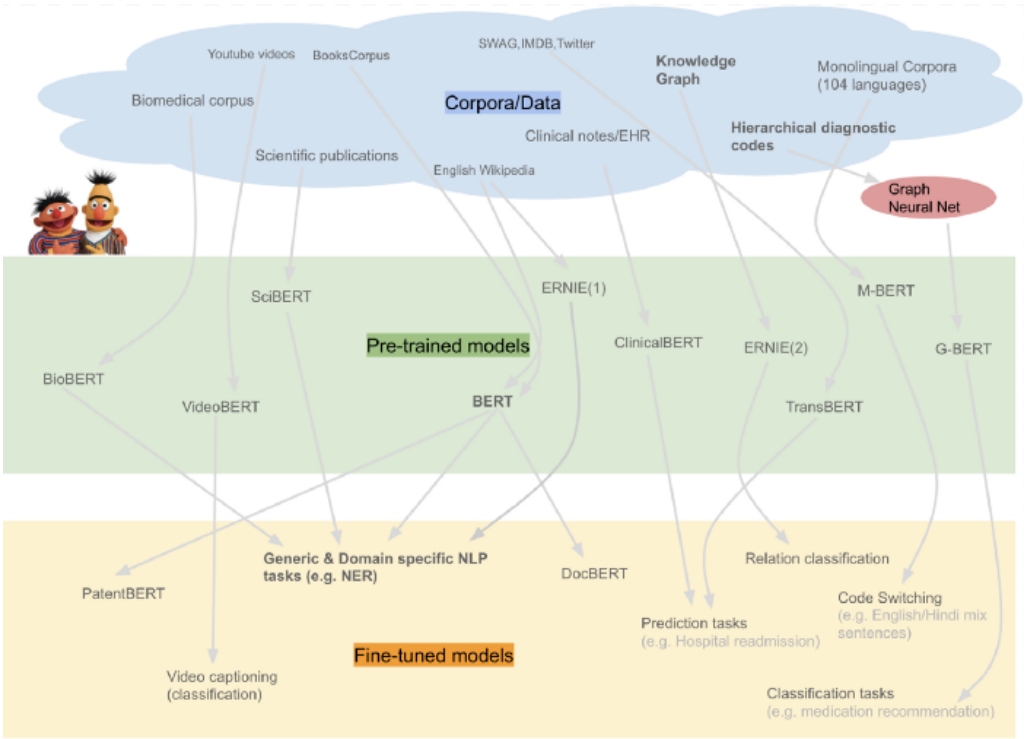


Figura 2.10: Varianti di BERT[25]

2.3.4.4 sBERT

Come detto, BERT è eccezionale nei compiti di somiglianza testuale semantica, il problema è che richiede molto tempo per corpus voluminosi (es. ∼65 ore per 10.000 frasi!) poiché richiede che entrambe le frasi[6] vengano immesse nella rete e questo aumenta notevolmente il tempo di addestramento. Per ovviare a questo problema è stato creato *sBERT*[5] o *Sentence-BERT*, una modifica del modello BERT che utilizza strutture di rete siamesi e triplette e aggiunge un'operazione di pooling all'output di BERT per ottenere embedding di dimensioni fisse di frasi semanticamente significative. Riprendendo l'esempio del "riso" di Sezione 2.2.4, al contrario dei metodi visti in precedenza detti *context-free* in cui l'embedding di una parola rimane invariato, a prescindere dal contesto, con modelli come BERT l'embedding varia a seconda del contesto in cui si trova la parola: sfruttando i meccanismi di attention che generano

[6]Si intendono le frasi di entrambe le lingue utilizzate per l'addestramento

contextual embedding, il modello genererà un determinato embedding per la frase "Mi passeresti il *riso*?" ed uno completamente differente per "Ho *riso* davvero tanto ieri sera". Questi embedding di frasi possono essere utilizzati per il confronto della somiglianza tra frasi (ad esempio utilizzando la cosine similarity), il clustering e la ricerca semantica, fornendo risultati più accurati e pertinenti.

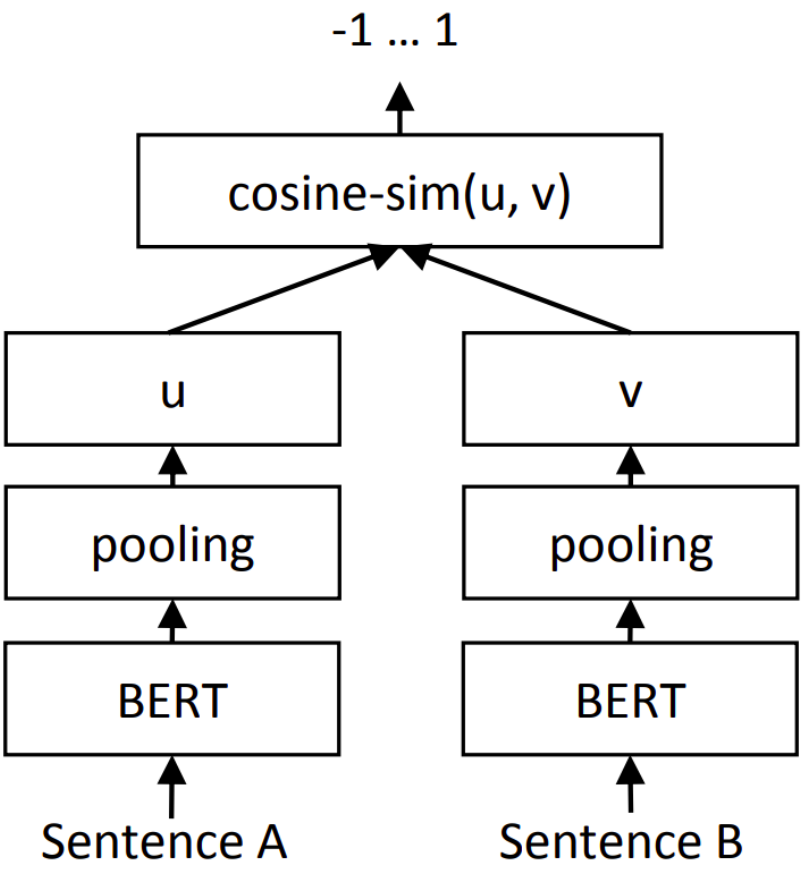


Figura 2.11: Architettura di sBERT per eseguire task di similarità[5]

2.4 Document embedding

Se volessimo limitarci a confrontare frasi, i sentence embedding sarebbero sufficienti, in quanto rappresentazione densa di frasi. Tuttavia, spesso occorre confrontare documenti, ovvero insiemi composti da più frasi, e i sentence embedding potrebbero risultare non abbastanza accurati nella rappresentazione di interi documenti. Nel nostro caso però, nonostante capiti che una segnalazione contenga più di una frase, data la loro brevità e il loro contesto, si è deciso di trattare una segnalazione come un'unica frase continuando ad utilizzare i sentence embedding. Osserviamo infatti come, nonostante siano presenti alcuni documenti piuttosto lunghi arrivando fino a 761 parole, il 95° percentile sulla lunghezza dei documenti si attesti a 65 parole con una mediana (o 2° quantile) di 17 parole:

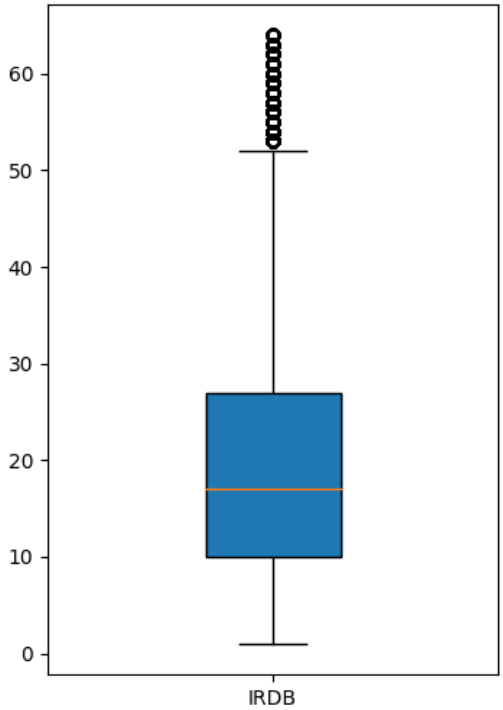


Figura 2.12: 95° percentile delle lunghezze delle segnalazioni

2.5 Metriche di confronto

Dopo aver scelto il modo più efficace per la costruzione dei propri embedding, occorre definire una metodologia per il confronto di questi, in modo da stabilire se e quanto due vettori siano simili. Allo stato dell'arte troviamo principalmente 3 metodologie di confronto:

- Word mover's distance
- Euclidean distance
- Cosine similarity

Va notato che, di queste tecniche, la *Word mover's distance* è applicabile solamente alle frasi, mentre le restanti sono applicabili a qualsiasi tipo di vettore di embedding.

2.5.1 Word mover's distance

Letteralmente "*distanza del passaparola*", è una tecnica che utilizza i word embedding delle parole dei due testi per misurare la distanza minima che le parole in un testo devono "percorrere" nello spazio semantico per raggiungere le parole nell'altro testo. Sebbene questo approccio abbia proprietà decisamente peculiari ed interessanti:

- È semplice da comprendere e utilizzare
- È altamente interpretabile in quanto la distanza tra due documenti può essere scomposta e spiegata osservando come le distanze ridotte tra le singole parole determinino il grado si somiglianza delle frasi
- Basandosi sui word embedding, non necessita di sentence embedding per confrontare due frasi

Tuttavia, presenta problematiche e difetti intrinseci nel suo funzionamento

- Valuta frasi senza utilizzare i sentence embedding, ereditando dunque le problematiche discusse in Sezione 2.2.4
- Necessita che le due frasi siano sintatticamente simili in modo da poterle confrontare parola per parola, nonostante due frasi possano avere significato simile pur essendo strutturalmente molto diverse

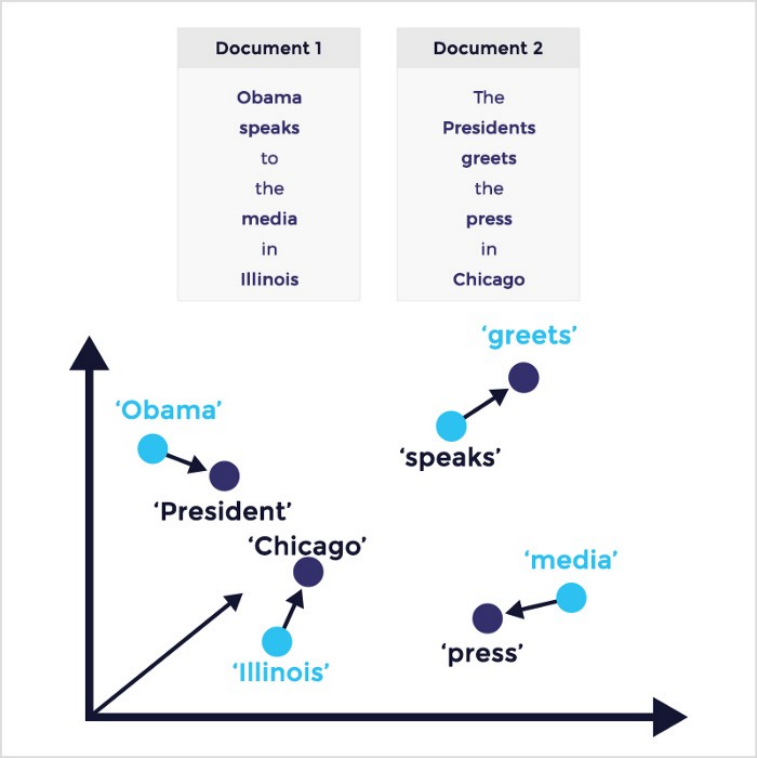


Figura 2.13: Word mover's distance[20]

2.5.2 Euclidean distance

La distanza euclidea tra due punti è la lunghezza del percorso che li collega. Viene calcolata con la formula

$$d = \sqrt{\sum_{i=1}^{k}(x_i - y_i)^2}$$

in cui vengono sommate le distanze tra le singole coordinate dei due vettori. Il teorema di Pitagora è un caso particolare di questa formula.

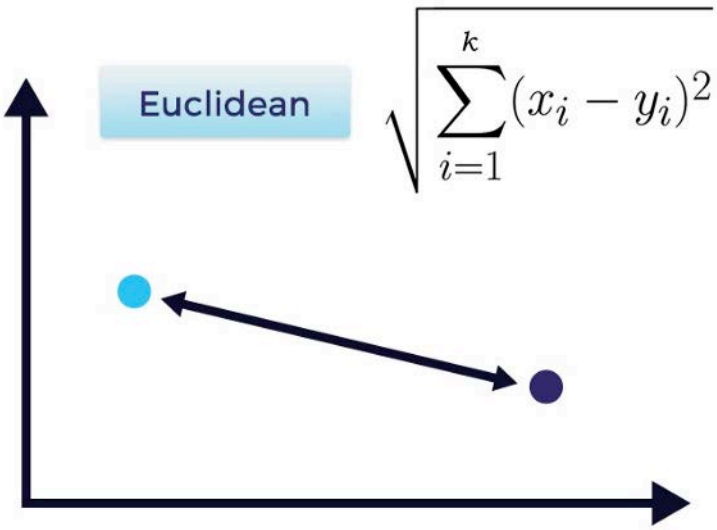


Figura 2.14: Euclidean distance[20]

2.5.3 Cosine similarity

È il metodo più utilizzato per confrontare due vettori. Si tratta di un prodotto scalare tra due vettori attraverso il quale è possibile trovare il coseno dell'angolo che li separa. Otterremo dunque un valore che va da 0, per vettori ortogonali, completamente scorrelati, ad 1 per vettori sovrapposti e quindi altamente simili, la cui distanza misurata come angolo è pari a 0 ($cos(0) = 1$).

$$d(A, B) = \cos\theta = \frac{A \cdot B}{\|A\|\|B\|}$$

Questa sarà la nostra metrica di riferimento.

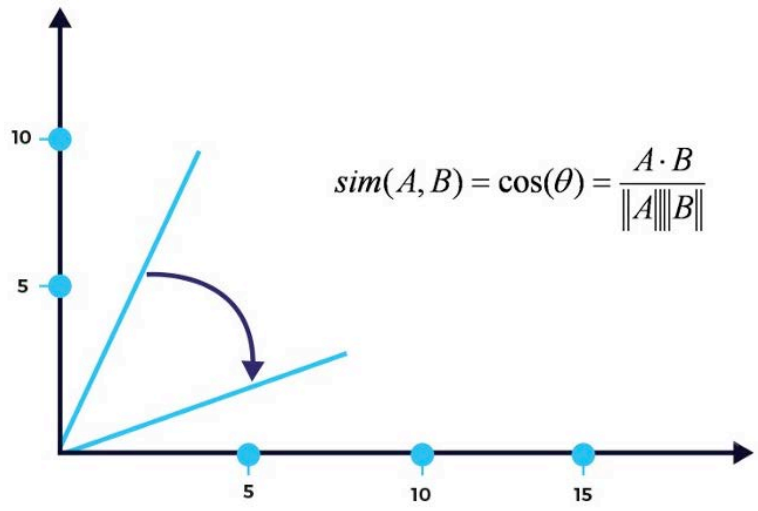


Figura 2.15: Cosine similarity[20]

2.5.4 Ranking

Oltre ai metodi appena discussi, vi sono metriche che misurano quanto i risultati di ricerca siano pertinenti; per farlo, ricorrono al *Relevance Feedback*, una tecnica che esamina i risultati ottenuti da una ricerca e ne rivaluta la rilevanza basandosi sul feedback relativo alla loro pertinenza ricevuto dall'utente. Il Relevance Feedback si divide in feedback esplicito, implicito e pseudo.

2.5.4.1 Explicit Relevance Feedback

Il feedback esplicito è ottenuto da valutatori di pertinenza che indicano la pertinenza di un documento recuperato per una query; questi possono essere binari (si/no) o graduati (valutazione a stelle). Questo tipo di feedback è definito esplicito solo quando i valutatori (o altri utenti di un sistema) sanno che il feedback fornito viene utilizzato come giudizio di rilevanza. Le informazioni di feedback sulla pertinenza devono essere interpolate con la query originale per migliorare le prestazioni della ricerca. Due metodi di misurazione delle prestazioni appartenenti a questa categoria e divenuti abbastanza conosciuti sono *Precision@k* e *NDCG* (*Normalized Discounted*

Cumulative Gain). Precision@k è la classica precision $\frac{TP}{TP+FP}$[7] ma calcolata solo sui primi k elementi, ovvero

$$P@k = \frac{TP}{k}$$

Questo approccio offre il suo contributo in situazioni in cui, ad esempio, si hanno 200 nuovi elementi da esaminare ogni giorno, ma il proprio organico ne può processare solo 100 al giorno; in questo caso occorre assicurarsi che gli elementi più promettenti siano tra i primi 100 in modo che possano essere analizzati. Utilizzando $P@100$ si riesce ad avere una metrica che indichi quanto "buoni" sono i primi 100 elementi. NDCG misura l'utilità, o il guadagno, di un documento in base alla sua posizione nell'elenco dei risultati. In questo modo, i documenti altamente rilevanti che appaiono in fondo in un elenco di risultati di ricerca saranno penalizzati. Il DCG corrispondente alla posizione p è

$$DCG_p = \sum_{i=1}^{p} \frac{rel_i}{log_2(i+1)} = rel_1 + \sum_{i=2}^{p} \frac{rel_i}{log_2(i+1)}$$

dove rel è la rilevanza del documento. Una formula alternativa per il DCG che pone una maggior enfasi nel recuperi di documenti rilevanti è

$$DCG_p = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{log_2(i+1)}$$

Quest'ultima formula è comunemente utilizzata nell'industria, comprese le principali società di ricerca Web e le piattaforme di competizione in ambito data science come Kaggle. L'NDCG normalizza il DCG utilizzando un DCG ideale detto IDCG (*Ideal Discounted Cumulative Gain*). Un $IDCG_p$ corrisponde quindi ad un elenco correttamente ordinato di documenti fino all'ordine p, quindi

$$NDCG_p = \frac{DCG_p}{IDCG_p}$$

dove

$$IDCG_p = \sum_{i=1}^{|REL_p|} \frac{rel_i}{log_2(i+1)}$$

[7] TP = True Positive, FP = False Positive

e REL_p rappresenta la lista dei documenti rilevanti (ordinati per rilevanza) nel corpus, fino alla posizione p.

2.5.4.2 Implicit Relevance Feedback

Il feedback implicito viene dedotto dal comportamento dell'utente, ad esempio, annotando quali documenti vengono selezionati (logs) o la durata del tempo trascorso a visualizzare un documento. Le principali differenze tra il feedback di rilevanza implicita e quello esplicito sono che l'utente non sta valutando la pertinenza a beneficio del sistema IR, ma sta solo soddisfacendo le proprie esigenze, e che l'utente non è necessariamente informato che il suo comportamento sarà utilizzato come feedback di pertinenza. Un esempio applicativo di feedback implicito è il *dwell time*, ovvero il tempo speso da un utente su un documento dal momento in cui clicca il relativo link sulla pagina fornita da un motore di ricerca a quando torna alla lista dei risultati. Si tratta di un importante indicatore di rilevanza in quanto nel caso in cui il dwell time risultasse essere troppo corto (rispetto a quanto stimato[8]), significa che probabilmente il documento in questione non è abbastanza importante o pertinente per la ricerca eseguita.

2.5.4.3 Pseudo Relevance Feedback

La PRF cerca di automatizzare la parte manuale intrinseca del feedback esplicito assumendo che i primi k elementi ottenuti dalla query di ricerca siano rilevanti. Con questa supposizione, non restituisce i risultati in questione all'utente, bensì li utilizza per estrarre parole chiave, utilizzando ad esempio TF-IDF, con cui espandere la query, che viene quindi eseguita nuovamente e i cui risultati, stavolta, vengono restituiti all'utente.

[8]Stima basata su lunghezza del documento, tipologia, dwell time medio, ...

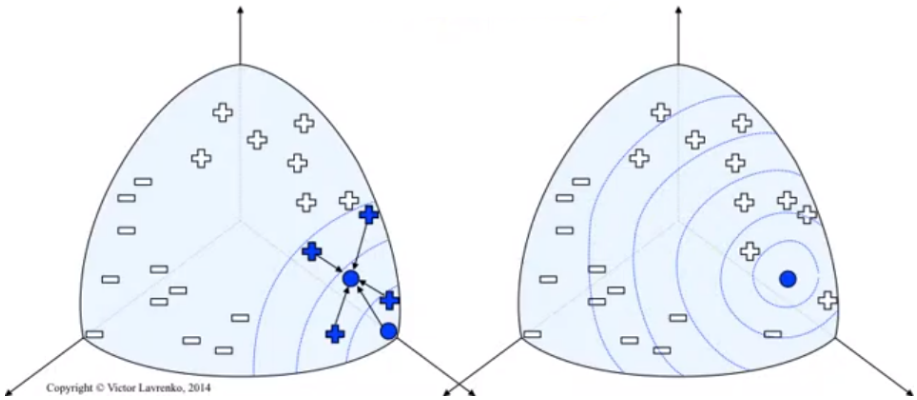


Figura 2.16: PRF: prima query (sinistra) e seconda query con l'aggiunta di keywords (destra)

In Figura 2.16 possiamo osservare come il "centroide" della query si sposti verso il cluster di "+" dopo aver effettuato l'espansione della query via PRF, fornendo quindi risultati più inerenti per primi.

Capitolo 3

Architettura

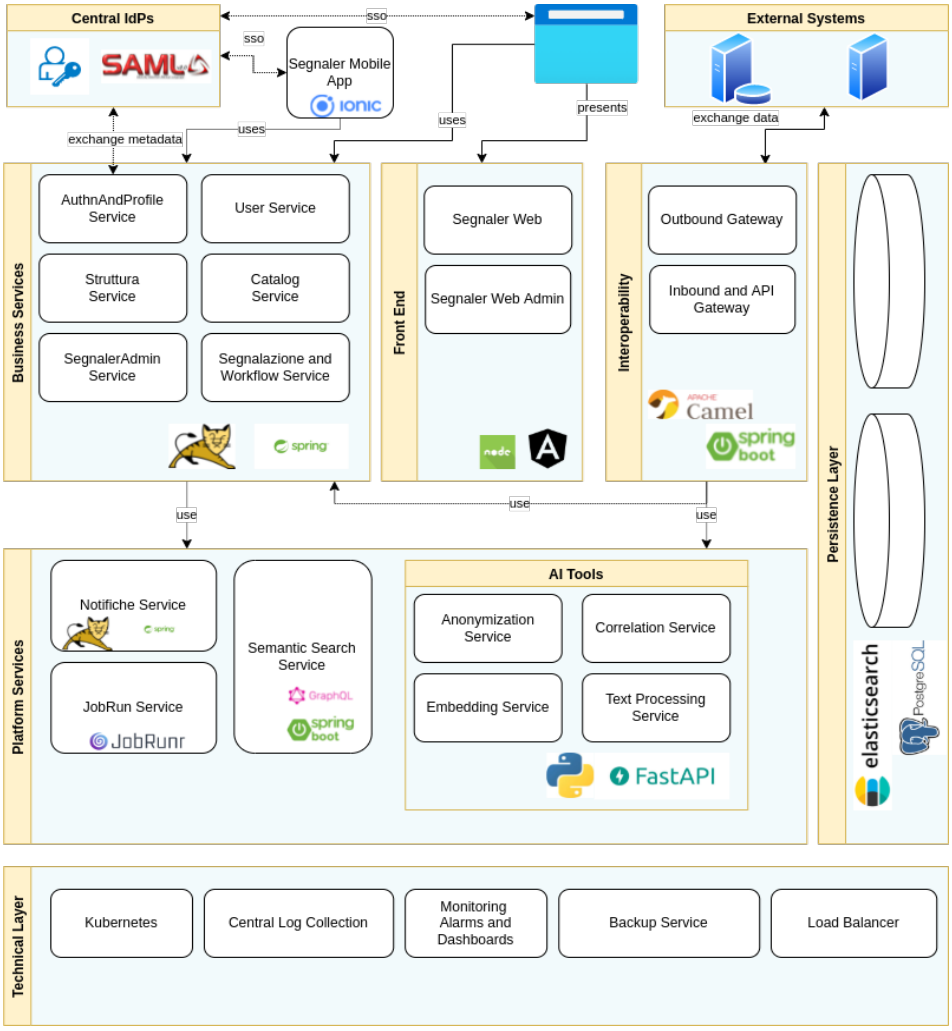


Figura 3.1: Architettura di SegnalER (schema logico)

La modularità di SegnalER ci consente di sorvolare sui dettagli dell'architettura dell'intera soluzione per concentrarci su quelli propri del motore di ricerca semantico.

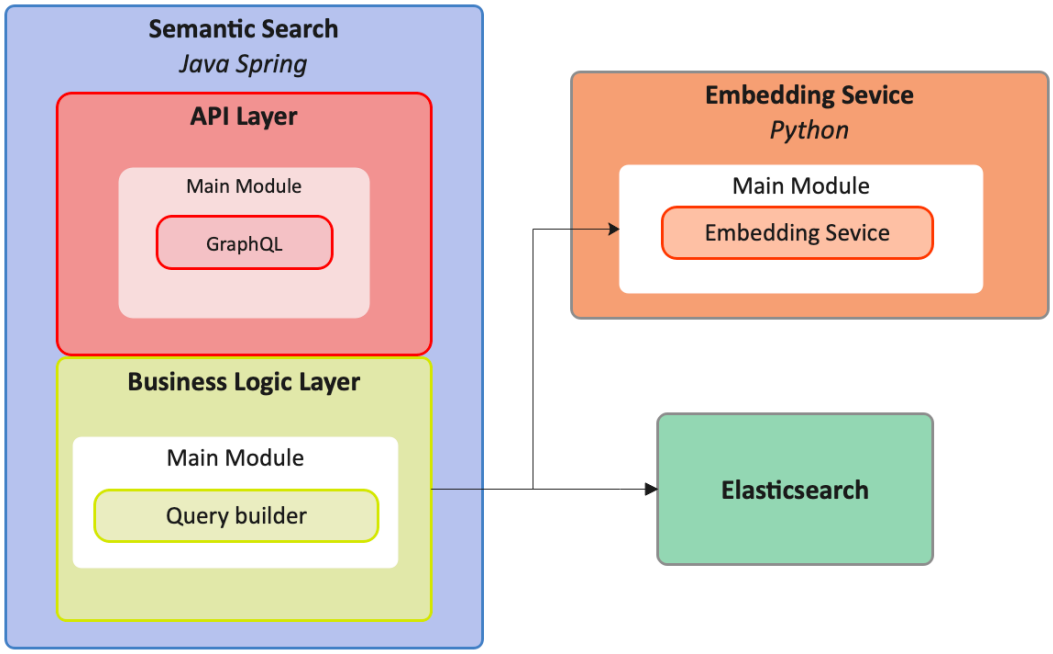


Figura 3.2: Architettura del motore di ricerca semantico

Il motore di ricerca semantico è stato scritto in Spring per Java ed è diviso in 2 moduli: *API* e *Business Logic*.

3.1 API

Lo strato di API contiene essenzialmente un endpoint GraphQL[1] per consentire di interrogare il servizio. La scelta dell'utilizzo di GraphQL è dovuta all'utilizzo *general purpose* per cui vuole predisporsi il servizio: l'idea è che più moduli/servizi interroghino il motore di ricerca, ognuno con le proprie esigenze; di conseguenza è stato scelto di adottare GraphQL per consentire ai vari client di modellare la propria richiesta secondo le proprie esigenze, evitando così di dover creare

[1] https://graphql.org/

e mantenere tutta una serie di API REST specifiche per ogni nuovo client che volesse usufruire del sevizio.

3.1.1 GraphQL

GraphQL, creato da Facebook nel 2012 e mantenuto dalla GraphQL Foundation dal 2019, è un query language per API che si propone come alternativa alle API REST e SOAP. Come REST e SOAP, GraphQL non è specifico per una sola piattaforma e si pone tra server e client aiutando a recuperare i dati in modo più organizzato. In GraphQL, le API vengono definite attraverso uno schema a grafo rappresentante le relazioni tra le varie entità del dominio. Questo grafo espone le capacità del server, le quali possono essere richieste dal client attraverso una query di introspezione, in modo tale che il client sia in grado di sapere in che forma e quali dati il server è in grado di fornire.

```
type Query {
    bookById(id: ID): Book
}

type Book {
    id: ID
    name: String
    pageCount: Int
    author: Author
}

type Author {
    id: ID
    firstName: String
    lastName: String
}
```

Listing 3.1: Esempio di schema GraphQL

In questo esempio vediamo come il server offra la possibilità di effettuare la query *bookById()* specificando l'id del libro interessato e restituisca un oggetto di tipo *Book* definito nello schema come un oggetto composto da *id*, *title*, *pageCount* e *author*. In GraphQL esistono alcuni tipi chiamati *scalari* che sono considerati i tipi base e consistono in *Int*, *Float*, *String*, *Boolean* e *ID* e tipi complessi che vanno definiti nello schema. Il tipo Book presenta attributi sia semplici che complessi, infatti, di seguito alla definizione di Book abbiamo quella di *Author*, composto interamente da campi semplici. Dal momento che GraphQL non è un database ma un servizio intermediario, è necessario implementare nel proprio server un *translating runtime layer*, ovvero uno strato che faccia da interprete tra GraphQL e il backend.

```
@Component
public class GraphQLProvider {

    @Value("classpath:schema.graphqls")
    private Resource graphQlSchema;
    private GraphQL graphQL;
    @Autowired
    GraphQLDataFetchers graphQLDataFetchers;

    @Bean
    public GraphQL graphQL() {
        return graphQL;
    }

    @PostConstruct
    public void init () throws IOException {
        String sdl = new String(graphQlSchema.getInputStream().readAllBytes());
        GraphQLSchema graphQLSchema = buildSchema(sdl);
        this.graphQL = GraphQL.newGraphQL(graphQLSchema).build();
    }

    private GraphQLSchema buildSchema(String sdl) {
        TypeDefinitionRegistry typeRegistry = new SchemaParser().parse(sdl);
        RuntimeWiring runtimeWiring = buildWiring();
        SchemaGenerator schemaGenerator = new SchemaGenerator();
        return schemaGenerator.makeExecutableSchema(typeRegistry, runtimeWiring);
    }

    private RuntimeWiring buildWiring() {
        return RuntimeWiring.newRuntimeWiring()
                .type(newTypeWiring("Query")
```

```
                .dataFetcher("bookById", graphQLDataFetchers.getBookByIdDataFetcher()))
            .type(newTypeWiring("Book")
                .dataFetcher("author", graphQLDataFetchers.getAuthorDataFetcher()))
            .build();
    }
}
```

Listing 3.2: Esempio di classe per l'interpretazione delle query GraphQL

Questa classe ha il compito di interpretare lo schema verificandone la correttezza ed esponendo gli endpoint descritti nello stesso. In particolare, si divide in 2 metodi principali, *buildSchema()* che si occupa di rendere lo schema eseguibile per il server e *buildWiring()* che si occupa di accoppiare endpoint GraphQL con metodi Java, detti *DataFetcher*. Un Data Fetcher ha il compito di eseguire la richiesta effettuata tramite query, ovvero, fornire i dati richiesti o effettuare le modifiche richieste.
Nella classe in Listing 3.3 notiamo 2 fetcher:

- **getBookByIdDataFetcher:** restituisce un libro dato il suo ID
- **getAuthorDataFetcher:** restituisce l'autore appartenente al libro contenuto nella richiesta, dal momento che con il primo fetcher viene restituito *authorId* che è un ID, non un Author, GraphQL provvederà automaticamente ad eseguire una query per cercare di recuperare l'autore dato il suo contesto (Book)

Se non viene specificato un Data Fetcher, verrà utilizzato quello di default che, ad esempio, provvede a creare oggetti JSON a partire da Map Java

```
@Component
public class GraphQLDataFetchers {

    private static final List<Map<String, String>> books = Arrays.asList(
            Map.of("id", "book-1",
                    "name", "Harry Potter and the Philosopher's Stone",
                    "pageCount", "223",
                    "authorId", "author-1"),
            Map.of("id", "book-2",
                    "name", "Moby Dick",
                    "pageCount", "635",
                    "authorId", "author-2")
```

```
    );

    private static final List<Map<String, String>> authors = Arrays.asList(
            Map.of("id", "author-1",
                    "firstName", "Joanne",
                    "lastName", "Rowling"),
            Map.of("id", "author-2",
                    "firstName", "Herman",
                    "lastName", "Melville")
    );

    public DataFetcher getBookByIdDataFetcher() {
        return dataFetchingEnvironment -> {
            String bookId = dataFetchingEnvironment.getArgument("id");
            return books
                    .stream()
                    . filter (book -> book.get("id").equals(bookId))
                    . findFirst ()
                    .orElse(null);
        };
    }

    public DataFetcher getAuthorDataFetcher() {
        return dataFetchingEnvironment -> {
            Map<String,String> book = dataFetchingEnvironment.getSource();
            String authorId = book.get("authorId");
            return authors
                    .stream()
                    . filter (author -> author.get("id").equals(authorId))
                    . findFirst ()
                    .orElse(null);
        };
    }
}
```

Listing 3.3: Esempio di classe DataFetcher per GraphQL

A questo punto il server GraphQL è pronto per essere interrogato.

```
{
    bookById(id: "book-1") {
        name
        pageCount
        author {
            id
            firstName
            lastName
        }
    }
}
```

```
{
  "data": {
    "bookById": {
      "name": "Harry Potter and the Philosopher's Stone",
      "pageCount": 223,
      "author": {
        "id": "author-1",
        "firstName": "Joanne",
        "lastName": "Rowling"
      }
    }
  }
}
```

Figura 3.3: Esempio di query (sinistra) e risposta (destra) GraphQL

Come ben visibile dalla risposta, la struttura è esattamente quella richiesta. Dopo questa breve panoramica viene spontaneo chiedersi quali vantaggi comporti implementare GraphQL piuttosto che normali API REST. Innanzitutto, avendo uno schema fortemente tipizzato ed introspezionabile, il frontend è facilitato nella stesura delle query e la documentazione delle API diventa di supporto, ma non essenziale. Inoltre, riesce a risolvere alcuni dei problemi più comuni di REST:

- **Assenza di over-fetching e under-fetching:** l' over-fetching si ha quando la risposta contiene più dati di quanti siano effettivamente necessari al richiedente; questo ha effetto sulle prestazioni sia in termini di tempo che in termini di dati trasmessi. Al contrario, l'under-fetching si ha quando la risposta contiene troppi pochi dati e risulta quindi obbligatorio effettuare più chiamate per recuperare tutti i dati necessari. In GraphQL questo non è un problema in quanto è possibile richiedere esattamente e solo i dati che interessano.

- **Risparmio su tempo e banda:** GraphQL consente di effettuare richieste multiple nella stessa query, il che porta a salvare banda e tempo riducendo il numero di query da effettuare.

- **Modellazione della risposta:** Il client sa a priori la struttura dati che riceverà dal client, poichè questa viene specificata in fase di interrogazione

- **Gestione delle versioni:** A causa di cambi nelle risorse o nelle richieste da parte dei fruitori delle API, uno sviluppatore può trovarsi costretto a creare più versioni delle stesse API REST (es. api.domain.com/v1/, api.domain.com/v2/) rischiando di creare confusione all'aumentare del numero di versioni. In GraphQL invece non è necessario creare e mantenere diverse versioni: l'URL e la query rimangono le stesse, semplicemente saranno aggiunti nuovi campi agli oggetti interessati dall'aggiornamento. Questo approccio risulta intuitivo dal momento che, a seguito di un aggiornamento, il client riceverà una warning se si troverà ad utilizzare dei campi deprecati

3.2 Business Logic

Il livello di Business Logic contiene tutta la parte algoritmica inerente al funzionamento del servizio, in questo caso contiene la logica di traduzione da richiesta del livello di API a query per *Elasticsearch*[2]. È estremamente importante che il livello di API e di BL siano ben distinti e che non vi siano dipendenze di uno dall'altro poiché, in questo modo è possibile cambiare o aggiungere tecnologie in grado di fornire API senza alterare in alcun modo lo strato di BL. Per ottenere questa modularità è necessario che lo strato di BL esponga una o più interfacce che verranno utilizzate dal livello di API: ogni tecnologia del livello di API, una volta ricevuta una richiesta, deve tradurla nell'oggetto richiesto dall'interfaccia esposta dal livello di BL; in questo specifico caso, lo strato di interpretazione del server GraphQL, ovvero il translating runtime layer, si occuperà di tradurre la richiesta GraphQL in un oggetto *ElasticSearchRequest* che sarà poi a sua volta convertito in una query interpretabile da Elasticsearch. Una volta ricevuta la richiesta, lo strato di BL controlla se è stata richiesta una ricerca classica, *exact match*, o

[2]https://www.elastic.co/

una ricerca semantica. Nel primo caso inoltra direttamente la richiesta alla classe che si occupa di costruire la query per Elasticsearch, altrimenti invia una richiesta di embedding contenente il testo della ricerca all'*Embedding Service*.

3.3 Embedding Service

L'Embedding Service è un servizio scritto in Python che espone API REST in grado di fornire sentence embedding a partire dalle frasi ricevute, un embedding per ogni frase. Si tratta di un servizio indipendente che mantiene caricato in RAM un modello addestrato in precedenza in grado di fornire sentence embedding facendo inferenza sulle frasi fornite.

3.4 Elasticsearch

Che la ricerca sia classica o semantica, l'iter che la richiesta deve fare terminerà comunque con Elasticsearch. Come base per il motore di ricerca semantico si è pensato di partire da un motore di ricerca preesistente che consentisse di effettuare query complesse contenenti, fra gli altri parametri, un criterio di selezione dei risultati personalizzabile, in modo da poter inserire il nostro algoritmo di ricerca semantica. La scelta è inevitabilmente ricaduta su **Elasticsearch**, famoso motore di ricerca basato sulla libreria **Lucene** di Apache. Elasticsearch converte Lucene in un sistema/motore di ricerca distribuito per fornire scalabilità orizzontale ed altri servizi come pool di thread, code, API di monitoraggio di nodi/cluster, API di monitoraggio dei dati, gestione dei cluster, ecc. In breve, estende Lucene e fornisce funzionalità aggiuntive. Elasticsearch ospita i dati su nodi di dati, ogni nodo ospita uno o più indici e ogni indice è diviso in elementi con ogni elemento che contiene parte dei dati dell'indice. Ogni elemento creato in Elasticsearch è un'istanza o un processo Lucene separato.

3.4.1 Elasticsearch: indici

An index is like a 'database' in a relational database.
It has a mapping which defines multiple types.
An index is a logical namespace which maps to one or more primary shards and can have zero or more replica shards.

ELASTICSEARCH

In Elasticsearch i documenti sono organizzati in indici e ogni indice possiede un proprio mapping in cui è definita la struttura del singolo documento. Se volessimo fare un confronto con un normale database, potremmo grossolanamente confrontarli come

- MySQL → Databases → Tabelle → Colonne/Righe
- Elasticsearch → Indici → Tipi → Documenti con Attributi

```
{
  "mappings": {
    "properties": {
      "age":    { "type": "integer" },
      "email":  { "type": "keyword" },
      "name":   { "type": "text"  }
    }
  }
}
```

Esempio di mapping

Un cluster Elasticsearch può contenere più indici (database), che a loro volta contengono più tipi (tabelle). Questi tipi contengono più documenti (righe) e ogni documento ha più proprietà (colonne).
NB: è importante notare che non è necessario specificare un mapping per un indice; infatti è possibile anche specificarlo solo per alcuni campi del documento o non specificarlo affatto: in questi casi sarà Elasticsearch ad aggiungere il campo al mapping in base al tipo di dato che ritiene più adatto.

3.4.2 Elasticsearch: ricerca

Elasticsearch utilizza una tipo di indicizzazione chiamato ad *indice inverso* che supporta ricerche full-text molto veloci. Un indice inverso elenca ogni parola univoca che appare in qualsiasi documento e identifica tutti i documenti in cui si trova ogni parola. In pratica, in fase di indicizzazione, Elasticsearch crea un indice composto da tutte le parole univoche contenute in ogni documento e ad ognuna di esse associa la lista degli ID dei documenti in cui compare.

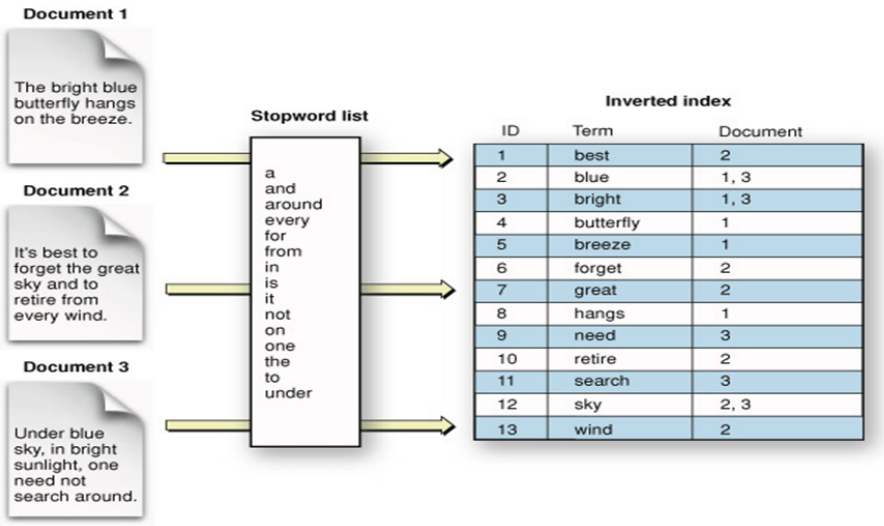


Figura 3.4: Elasticsearch: esempio di indice inverso

Questo processo viene gestito da un analizzatore, il quale si occuperà di dividere il testo nei tokens che saranno aggiunti all'indice inverso. I criteri di suddivisione del testo in tokens varia da analizzatore ad analizzatore. Elasticsearch ne fornisce diversi built-in ma offre anche la possibilità di utilizzare un proprio analizzatore *custom*. Nel nostro caso utilizzeremo il **Simple Analyzer**, il quale si limiterà a suddividere il testo in tokens ogni volta che incontrerà un carattere diverso da una lettera, premurandosi di rendere ogni lettera minuscola. Utilizzando questo meccanismo, è possibile recuperare un qualsiasi documento con velocità *near-realtime*[3].

[3]https://www.elastic.co/guide/en/elasticsearch/reference/current/near-real-time.html

3.4.2.1 Query di ricerca

Ogni ricerca effettuata su Elasticsearch, così come avviene per i database relazionali, si avvale dell'utilizzo di query strutturate e modulari per consentire la massima flessibilità di ricerca e filtraggio. Le query di ricerca per Elasticsearch sono dei JSON da utilizzare come body per richieste POST all'endpoint `/<index_name>/_search` del proprio server.

```
{
  "query": {
    "bool": {
      "must": [
        {
          "query_string":
          {
            "query": "text_here",
            "minimum_should_match": "100%"
          }
        }
      ],
      " filter ": [ ]
    }
  },
  " fields ": [],
  "_source": true,
  "size": 10,
  "from": 0
}
```

Listing 3.4: Esempio di query exact match per Elasticsearch

```
{
   "query": {
      "script_score": {
         "query": {
            "match_all": {}
         },
         "script": {
            "source": " cosineSimilarity (params.queryVector, 'text_vector') + 1.0",
            "params": {
               "queryVector": [
               ]
            }
         }
      }
   },
```

```
  "fields": [],
   "_source": true,
  "size": 10,
  "from": 0
}
```

Listing 3.5: Esempio di query semantica per Elasticsearch

La query in Listing 3.4 può essere utilizzata per fare ricerche puntuali basandosi sul testo immesso. In sostanza la query stabilisce che

- i risultati dovranno contenere il 100% del testo presente come valore del campo *query* di *query_string*
- saranno sottoposti ai filtri stabiliti in *filters*, in questo caso nessuno
- saranno restituiti solo i campi richiesti in *fields*, in questo caso nessuno
- sarà restituito l'intero elemento salvato nell'indice, dal momento che *_source* è *true*
- saranno restituiti solo i primi 10 elementi, ordinati di default da quello più pertinente a quello meno pertinente
- non vi è un offset iniziale tra i risultati da restituire, dal momento che *from* è *0*

Anche se a prima vista potrebbe sembrare poco utile saltare i primi n risultati migliori con il parametro *from*, questo si rivela particolarmente utile se utilizzato assieme a *size* per consentire di effettuare paginazione sui risultati. La query in Listing 3.5 viene invece utilizzata per effettuare ricerche semantiche. Al primo "livello" la query risulta uguale alla precedente, a cambiare è la composizione dell'oggetto *query*: al suo interno infatti notiamo come, a differenza di prima dove avevamo una query booleana contenente una *query_string* addetta a controllare che il testo corrispondesse in una determinata percentuale, ora vi sia uno *script_score*, ovvero una query in grado di eseguire codice e restituire uno score. Il codice in questione viene scritto nell'elemento *source* di *script* in un linguaggio proprietario di Elasticsearch chiamato *Painless*[4]

[4] https://www.elastic.co/guide/en/elasticsearch/reference/current/modules-scripting-painless.html

che presenta una sintassi simile a Java con qualche modifica per adattarlo meglio all'ambiente circostante e viene eseguito direttamente su JVM. In questo modo è possibile includere nella query la propria logica di match degli elementi dell'indice: il codice verrà eseguito su ogni elemento restituendo un punteggio ad ogni esecuzione, tale punteggio sarà poi utilizzato per determinare quali elementi restituire e quali no. In questo esempio, il codice

```
cosineSimilarity (params.queryVector, 'text_vector') + 1.0
```

Listing 3.6: Cosine similarity in Painless

calcola la cosine similarity tra il vettore contenuto nel parametro chiamato *queryVector* e quello appartenente alla proprietà *text_vector* dell'i-esimo elemento; il "+ 1.0" finale serve a mantenere positivo lo score (che deve necessariamente essere positivo per la logica di Elasticsearch) dal momento che la funzione *cosineSimilarity()* di Painless restituisce un valore che va da -1 a +1.

3.5 Integrazione in SegnalER

Tutta la complessità nell'effettuare ricerche su Elasticsearch è racchiusa all'interno del servizio di ricerca semantica, come mostrato nello schema di Figura 3.2, che espone il seguente schema GraphQL

```
scalar JSON
scalar Date

type Query {
    status: ElasticSearchStatus
    search(request: ElasticSearchRequest): ElasticSearchResult
}

type ElasticSearchStatus {
    module: String
    version: String
    dataSourceStatus: String
    elasticState: ElasticState
    dataSourceType: String
}
```

```
type ElasticState {
    name: String
    cluster_name: String
    cluster_uuid: String
    version: JSON,
    tagline: String
    statusCode: Int
}

type ElasticSearchResult {
    total: Int
    results: [Entry]
    aggregations: AggregationsResult
}

type Entry {
    id_segnalazione: Int
    score: Float
    code_segnalazione: String
    evento_date: String
    last_update_date: String
    schede_correlate: String
    segnalazione_status_description: String
    segnalazione_type_description: String
    segnalante: String
    unita_riferimento: String
}

type AggregationsResult {
    tipoSegnalazione: [Bucket]
    tipoScheda: [Bucket]
    aziendaSanitaria: [Bucket]
    data: [Bucket]
    idSegnalante: [Bucket]
    mostraSegnalazioniAnonime: [Bucket]
    idPersonaCoinvolta: [Bucket]
    statoSegnalazione: [Bucket]
}

type Bucket {
    key: String!
    docCount: Int!
}

input ElasticSearchRequest {
    index: String!
    text: String!
```

```
    modelName: String!
    from: Int
    size: Int
    exactMatch: Boolean
    filters: Filters
    aggregations: Aggregations
}

input Filters {
    tipoSegnalazione: [ID]
    tipoScheda: [ID]
    aziendaSanitaria: [Int]
    data: [Date]
    idSegnalante: [ID]
    mostraSegnalazioniAnonime: Boolean = false
    idPersonaCoinvolta: [Int]
    statoSegnalazione: [Int]
    jwtRoleFilter: JSON
}

input Aggregations {
    tipoSegnalazione: Boolean
    tipoScheda: Boolean
    aziendaSanitaria: Boolean
    data: Boolean
    dataFormato: String = "month"
    idSegnalante: Boolean
    mostraSegnalazioniAnonime: Boolean
    idPersonaCoinvolta: Boolean
    statoSegnalazione: Boolean
}
```

Listing 3.7: Schema GraphQL del Semantic Search Service

Attraverso l'API *search* è possibile effettuare una ricerca specificando i dettagli della richiesta nel parametro *request* di tipo *ElasticSearchRequest* e quelli della risposta nel valore di ritorno di tipo *ElasticSearchResult*.

```
query Search {
    search(request: {
        index: "idx_test"
        modelName: "wiki_idf_model.bin"
        text: "Errata segnalazione in lista operatoria del lato dell'intervento"
        exactMatch: false
        filters: {
            aziendaSanitaria: 4
            data: "2021-12-07"
```

```
            idSegnalante: [17, 78]
            tipoSegnalazione: "A01"
        }
        aggregations: {
            statoSegnalazione: true
            data: true
        }
    }) {
        total
        results {
            id_segnalazione
            score
        }
        aggregations {
            statoSegnalazione {
                ... allBucketFields
            }
            data {
                ... allBucketFields
            }
        }
    }
}

fragment allBucketFields on Bucket {
    key
    docCount
}
```

Listing 3.8: Esempio di ricerca utilizzando l'API *search* esposta da GraphQL

Nell'esempio in Listing 3.8 si può apprezzare a pieno la versatilità dei GraphQL che consente di fornire parametri strutturati e complessi e di specificare precisamente la struttura attesa dal server e di poterla eventualmente modificare a piacere senza dover effettuare alcuna richiesta di modifica API al team di sviluppo del server. In particolare, per questo esempio il servizio di ricerca semantica genererà la seguente query per Elasticsearch:

```
{
  "query":{
    "script_score":{
      "query":{
        "bool":{
          "must":[
            {
```

```
                "query_string":{
                    "query":"Errata segnalazione in lista operatoria del lato dell'intervento",
                    "minimum_should_match":1
                }
            }
        ],
        " filter ":[
            {
                "terms":{
                    "tipoSegnalazione":[
                        "A01"
                    ]
                }
            },
            {
                "terms":{
                    "aziendaSanitaria":[
                        "4"
                    ]
                }
            },
            {
                "terms":{
                    "data":[
                        "2021-12-07"
                    ]
                }
            },
            {
                "terms":{
                    "idSegnalante":[
                        "17",
                        "78"
                    ]
                }
            },
            {
                "term":{
                    "anonima":false
                }
            }
        ]
    }
},
"script":{
    "source":"0.5*_score + 0.5*(cosineSimilarity(params.queryVector, 'embedding') + 1.0)",
    "params":{
```

```
                "queryVector":[...]
            }
        }
    }
  },
  "fields":[
    "id_segnalazione",
    "score"
  ],
  "_source":false,
  "size":100,
  "from":0,
  "aggs":{
    "statoSegnalazione":{
      "terms":{
        "field":"statoSegnalazione",
        "size":65536
      }
    },
    "data":{
      "date_histogram":{
        "field":"date",
        "calendar_interval":"month"
      }
    }
  }
}
```

Listing 3.9: Query Elasticsearch generata dal servizio di ricerca semantica a partire dalla richiesta in Figura 3.8

Dove *queryVector* (qui omesso per questioni di spazio) conterrà una lista di 300 numeri float rappresentanti l'embedding della frase "Errata segnalazione in lista operatoria del lato dell'intervento" generato dal servizio di Embedding (vedi Figura 3.2) utilizzando il modello specificato nella richiesta[5]. Si può notare come questa query sia in realtà un mix di 2 query, infatti, a seguito di alcuni esperimenti e ricerche[9], si è giunti ad una query in cui una parte del punteggio attribuito all'i-esimo documento in fase di ricerca è dato da una query lessicale che si assicura che almeno una parola della frase oggetto della query sia contenuta nel documento in esame e da un'altra che calcola

[5]In realtà, una volta giunti in produzione, indice e modello saranno rimossi come parametri e resi statici

la cosine similarity tra l'embedding della frase e quello del documento. In aggiunta, nella query troviamo i filtri e le aggregazioni, così come il limite di elementi da restituire e l'offset. Il risultato restituito dal server GraphQL dopo aver ricevuto la risposta da Elasticsearch è il seguente


```
{
  "data": {
    "freeTextSearch": {
      "total": 100,
      "results": [
        {
          "id": 24,
          "score": 0.9567
        },
        {
          "id": 4125,
          "score": 0.7895
        },
        {
          "id": 4477,
          "score": 0.75007
        },
        {
          "id": 49473,
          "score": 0.6842
        },
        {
          "id": 17310,
          "score": 0.6431
        }
      ],
      "aggregations": {
        "statoSegnalazione": [
        ,
          {
            "key": "49852800000",
            "docCount": 29
          },
          {
            "key": "52531200000",
            "docCount": 15
          }
        ],
        "data": [
          {
            "key": "41904000000",
```

```
          "docCount": 42
        },
        {
          "key": "44582400000",
          "docCount": 8
        },
        {
          "key": "47174400000",
          "docCount": 1
        }
      ]
    }
  }
}
```

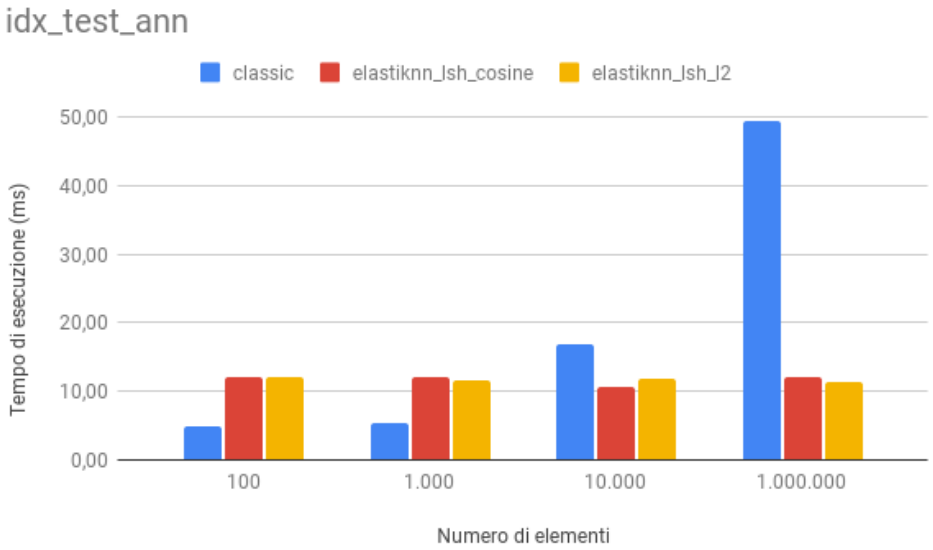
Listing 3.10: Risposta del servizio di ricerca semantica alla richiesta di Figura 3.8

3.5.1 Velocizzazione delle ricerche con Approximated Nearest Neighbor

Elasticsearch, basandosi sul suo sistema di indicizzazione ad indice inverso, riesce ad eseguire ricerche lessicali con tempistiche near-realtime anche su indici di grosse dimensioni. Ciò non vale però se la query è *scripted*, nel qual caso lo script specificato nella query andrà obbligatoriamente eseguito su ogni documento appartenente all'indice. Per ovviare a questo problema e cercare di ridurre i tempi di ricerca con query aventi script, è stato installato un plugin per Elasticsearch chiamato *Elastiknn*[6]. Elastiknn fornisce tipi di dato alternativi al *dense vector* utilizzato da Elasticsearch per indicizzare gli embedding, potendo così indicizzare i documenti in modo diverso da quanto fa Elasticsearch utilizzando proprio questi campi. Lo scopo di questo plugin è quello di velocizzare le ricerche semantiche utilizzando un Approximated Nearest Neighbor (ANN): il funzionamento si basa su LSH (Locality-Sensitive Hashing), un algoritmo che esegue l'hash dei documenti che riceve in input ma che, a differenza dei classici algoritmi di hashing che cercano di minimizzare le collisioni tra hash, punta a massimizzarle per poter raggruppare gli hash

[6] https://elastiknn.com/

simili in ”bucket” (il numero di bucket è molto più piccolo dell’universo dei possibili elementi di input); in questo modo si ha un’alta probabilità che elementi simili siano nello stesso bucket o in bucket vicini e che elementi molto diversi siano in bucket più lontani. Come suggerisce il nome, questo tipo di ricerca è approssimata, il che vuol dire che si accetta di non avere la certezza di ottenere i risultati migliori con la garanzia, però, che i tempi di esecuzione siano molto più ridotti. Di seguito, riportiamo un confronto tra una ricerca semantica classica, una effettuata con Elastiknn utilizzando l’algoritmo *Random Projection* per eseguire gli hash ed indicizzare i documenti in modo che supportino la cosine similarity come algoritmo di confronto ed una terza, sempre con Elastiknn, che utilizza il metodo Stable Distributions per hash e stoccaggio per consentire l’utilizzo di L2 (distanza Euclidea) come termine di paragone. Per ogni tipo di ricerca sono stati eseguiti 4 esperimenti, rispettivamente con 100, 1.000, 10.000 e 100.000 elementi in indice. I 4 esperimenti sono stati a loro volta eseguiti su 2 indici: *idx_test_ann* di 46.818 elementi e *urp_wiki_idf* di 362.368 elementi. Lo scopo alla base dell’utilizzo di 2 indici di dimensioni diverse è quello di offrire una visualizzazione più comprensibile dei grafici dato lo schiacciamento dei risultati delle prime ricerche in confronto all’ultima (nel primo indice infatti la query sarà eseguita solo su 46.818 elementi, non su 100.000).



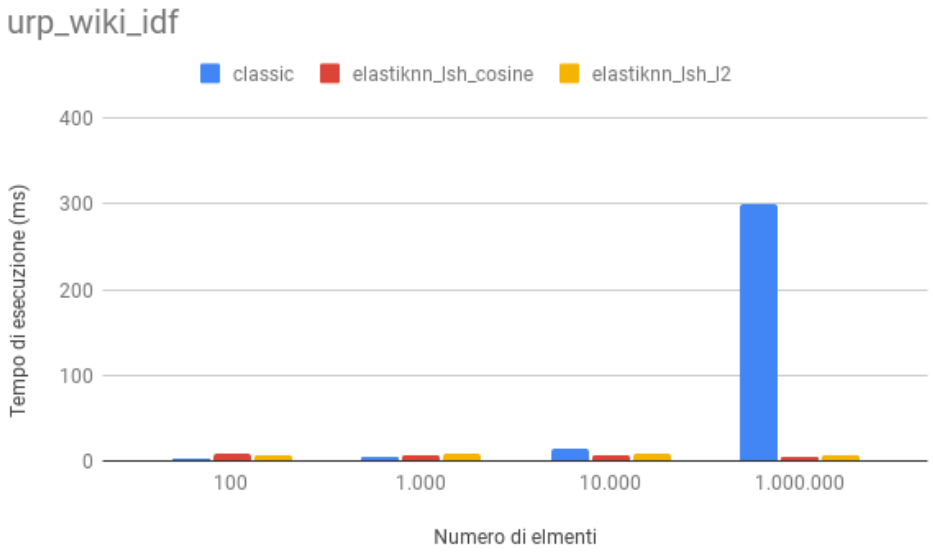


Figura 3.5: Valutazione Elastiknn sull'indice idx_test_ann (sopra) e urp_wiki_idf (sotto)

Dal confronto dei due grafici in Figura 3.5 osserviamo come su indici di dimensioni molto ridotte convenga utilizzare la ricerca classica mentre, già a partire da indici di 10.000 elementi, l'ANN impiega il 37,65% in meno con la cosine similarity e il 30,58% in meno con L2. La vera differenza, però,

si nota con indici di dimensioni maggiori, infatti, su un indice di 100.000 elementi, mentre la ricerca classica impiega 300,4 ms, Elastiknn impiega 5 ms con la cosine similarity e 7 ms con L2, si parla di rispettivamente del 0,016% e 0,023% del tempo impiegato dalla ricerca classica!

Capitolo 4

Fase di studio

La fase di studio, specie in un progetto di dimensioni notevoli o fortemente sperimentale come questo, basato su diversi componenti che fanno uso di NLP, rappresenta una parte fondamentale al fine di sviluppare una soluzione efficiente e al passo con il corrente stato dell'arte in questo settore. Una volta stabilito l'utilizzo di Elasticsearch come motore di ricerca e la cosine similarity come criterio di confronto per la parte semantica della ricerca, occorre stabilire come saranno creati gli embedding. Come descritto nel Capitolo 2 in modo abbastanza esaustivo ma non completo, esistono diverse tecniche di costruzione degli embedding, ma non esiste un criterio univoco e condiviso per determinare se una rappresentazione è meglio di un'altra. Se si trattasse di un task di classificazione avremmo indicatori come accuratezza, precisione, recall, ma trattandosi di un task di similarità tra frasi, la scelta di un modello piuttosto che un altro è guidata principalmente dalle esigenze del contesto in cui si trova e dalla percezione di "risultato corretto" da parte degli utilizzatori di tale modello. Nel caso di SegnalER serviva un modello che fosse in grado di interpretare correttamente frasi di lunghezza medio-corta e soprattutto che riuscisse a gestire parole OOV[1] in modo da essere resistente ad eventuali errori di battitura, non così rari effettuando una ricerca testuale. La scelta del modello tra quelli descritti in precedenza si è quindi ristretta

[1] Out Of Vocaboulary: fuori vocabolario, non incontrate in fase di addestramento

a 2 possibili candidati: FastText e sBERT. Data l'imponenza del modello sBERT, si è scelto di cominciare valutando FastText, per poi eventualmente confrontarlo con sBERT. Stabilito il modello per la costruzione degli word embedding, occorre decidere come costruire i sentence embedding ma, siccome non vi è un criterio universale per giudicare un modello addestrato per un task di similarità, si è deciso di utilizzare tutti e 3 gli approcci descritti in Sezione 2.3 in modo da poterli confrontare durante lo studio.

4.1 Dataset e problematiche

Il dataset a disposizione, estratto da uno storico di segnalazioni fornite dalla regione chiamato IRDB (Incident Reporting Database), contiene 46.851 segnalazioni aventi ciascuna diverse proprietà tra cui un testo, una classe, una data. Per quanto riguarda l'addestramento del modello sarà sufficiente il testo della segnalazione, ma la classe può essere utile per osservare il bilanciamento del dataset o per effettuare test di classificazione in modo da avere un parametro di confronto tra modelli più obiettivo dell'occhio critico di un utilizzatore. Tuttavia, questo dataset presenta diversi problemi, a partire dal forte sbilanciamento di classi per continuare con l'assenza di validation e test set che andranno estratti da questo.

4.1.1 Dati sporchi

Le segnalazioni presenti nell'IRDB sono vere segnalazioni immesse dal personale sanitario. Se da un lato questo è un bene perchè ci pone davanti al caso reale, dall'altro la presenza di punteggiatura, errori di battitura e parole molto rare (che compaiono meno di 5 volte in tutto il corpus) mettono in difficoltà l'apprendimento degli embedding. Per questo motivo è stato introdotto un preprocessing dei dati (cosa molto comune in ambito NLP) utilizzando il *simple_preprocess* di *Gensim*[2], una libreria Python per l'indicizzazione di

[2]https://github.com/RaRe-Technologies/gensim

documenti e la modellazione ed esecuzione di task di similarità con corpora di grandi dimensioni.

```
# Taking apart the sentences to embed and cleaning them
df = df.get(EMBED_FIELD_NAME).apply(lambda x: ' '.join(simple_preprocess(x)))
```

Listing 4.1: Pulizia delle descrizioni con simple_preprocess() di Gensim

4.1.2 Scarsità di dati

Specie se sbilanciate, 46.851 segnalazioni non sono tante per un task di questo tipo e il fatto che la lingua fosse quella italiana non ha di certo aiutato. Oltretutto, dato il tipo di segnalazioni, ovvero incidenti (intenzionali o meno) riguardanti i pazienti e segnalati dal personale sanitario, è molto comune trovare segnalazioni molto simili se non identiche, riducendo di fatto il numero di elementi del dataset effettivamente utili all'apprendimento. Vani sono stati i tentativi di trovare altri dataset con cui integrare il nostro o da utilizzare per metriche di confronto. Si è anche provato ad aumentare le dimensioni dell'IRDB facendo *data augmentation* utilizzando diverse tecniche tra cui doppie traduzioni (italiano → inglese → italiano) utilizzando le API di Google Translate[3], ma il dataset risultante non ha portato ad alcun miglioramento in quanto le segnalazioni risultavano troppo simili tra loro, non portando di fatto alcun valore aggiunto.

4.1.3 Interpretazione degli embedding

Un altro, non meno importante, problema riscontrato nello sviluppo di questo progetto è stato capire se gli embedding fossero effettivamente rappresentativi delle rispettive parole e fino a che punto[8], compito non immediato specie se ricordiamo che gli embedding hanno dimensione 300.

[3]https://cloud.google.com/translate

Figura 4.1: Rappresentazione 3D del dataset attraverso la riduzione dimensionale (con PCA) di ogni sentence embedding a 3 dimensioni

Da questa rappresentazione ottenuta con PCA possiamo notare come siano visibili 2 cluster nel dataset nonostante le classi di segnalazione siano 40. Ciò è dovuto ad un forte sbilanciamento che fa si che due classi prevalgano sulle altre, una delle quali è una classe generica che quindi, di fatto, comprende segnalazioni appartenenti ad altre classi.

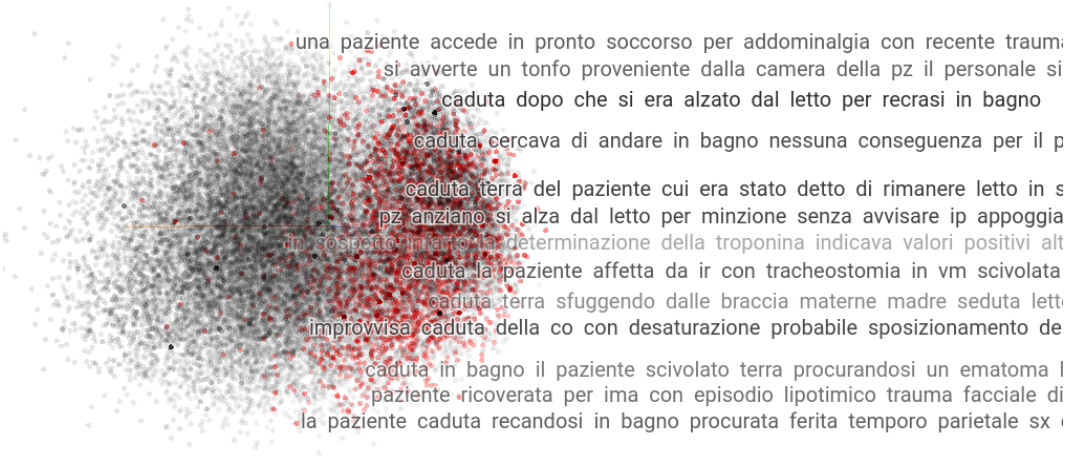


Figura 4.2: Distribuzione della parola ”caduta” (via embedding) nel dataset

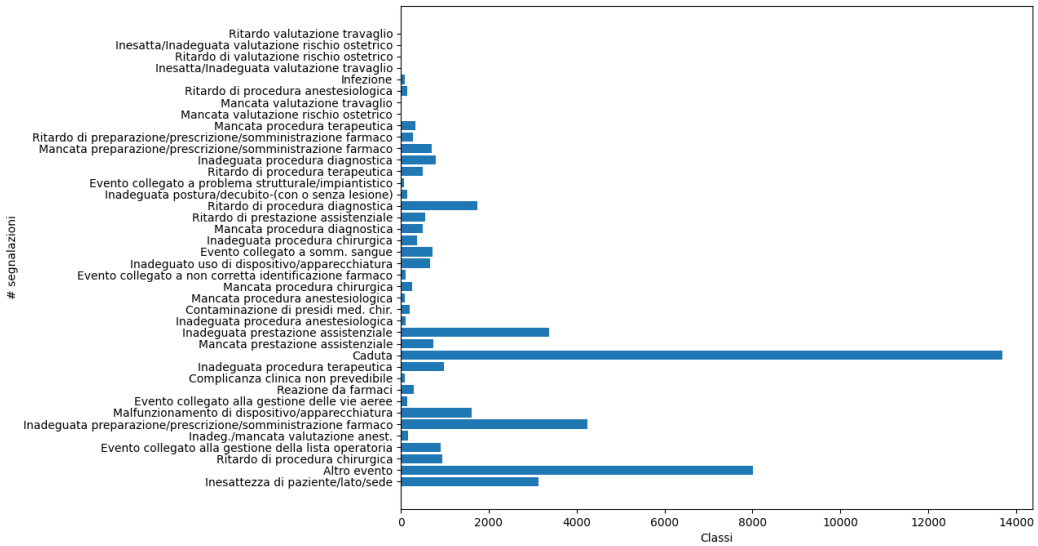


Figura 4.3: Istogramma delle classi facenti parte di IRDB

Il forte sbilanciamento assieme all'errata classificazione di numerose segnalazioni sotto la classe "Altro evento" rendono questo dataset non affidabile per un task di classificazione.

4.2 Sviluppo e confronto modelli

4.2.1 Word embedding

Sono stati realizzati diversi modelli prima di riuscire a trovare il più performante, complici i problemi discussi nella sezione precedente. Sono stati quindi eseguiti diversi esperimenti di data augmentation e tuning degli iperparametri delle reti, facendo prove di bilanciamento del dataset aumentando solo classi diverse da "caduta" e di tuning su learning rate, epoche e parametri propri di FastText (vedi modelli "*balanced*" e "*low learn*" in Figura 4.4 e 4.5), arrivando ad ottenere i parametri più efficienti e notando come un dataset aumentato con le tecniche sopracitate non portasse a miglioramenti evidenti ma si limitasse a "schiacciare" gli embedding, peggiorando di fatto i risultati. Sono quindi emersi 3 principali modelli di word embedding:

- Custom
- Wiki
- Wiki finetuned Custom

Il primo modello realizzato, chiamato *Custom*, è stato addestrato solamente con l'IRDB utilizzando il porting di FastText per Python[4].

```
HOME_DIR = Path(".")
MODELS_DIR = HOME_DIR/"models"
DATA_DIR = HOME_DIR/"datasets"
DATASET_NAME = "irdb_corpus.csv"
EMBED_FIELD_NAME = "text"
CLEAN_FILE_NAME = "IRDB_corpus_clean_event_descriptions.txt"

df = pd.read_csv(DATA_DIR/DATASET_NAME)

# Taking apart the sentences to embed and cleaning them
df = df.get(EMBED_FIELD_NAME).apply(lambda x: ' '.join(simple_preprocess(x, min_len=0,
                                                                          max_len=100)))

# Saving the sentences to file so that Fasttext can pick-up them
df.to_csv(MODELS_DIR/CLEAN_FILE_NAME,
          index = False,
          sep = ' ',
          header = False,
          quoting = csv.QUOTE_NONE,
          escapechar = " ")

# Training the model
model = fasttext.train_unsupervised(input=str(MODELS_DIR/CLEAN_FILE_NAME),
                                    lr=0.15, # learning rate (default 0.1)
                                    epoch=30, # default 5
                                    dim=300, # size of word vectors
                                    verbose=True)

# Saving the model
model.save_model(str(MODELS_DIR/"model_") + datetime.now().strftime("%Y%m%d%H%M%S") + ".bin")
```

Listing 4.2: Creazione- del modello *Custom*

Successivamente, per cercare di ovviare alla scarsità di dati, è stato utilizzato il modello *cc.it.300.bin*[5], un modello fornito direttamente da FastText, addestrato

[4]https://fasttext.cc/docs/en/python-module.html
[5]https://fasttext.cc/docs/en/crawl-vectors.html

sull'intera Wikipedia Italia assieme a Common Crawl[6]. Per affinare un modello altrimenti generico sulla lingua italiana, attraverso Gensim, è stato applicato un processo di fine tuning al modello precedente utilizzando l'IRDB. L'intento era quello di realizzare un modello che avesse la conoscenza generica della lingua italiana ma che fosse specializzato nel nostro ambito di utilizzo.

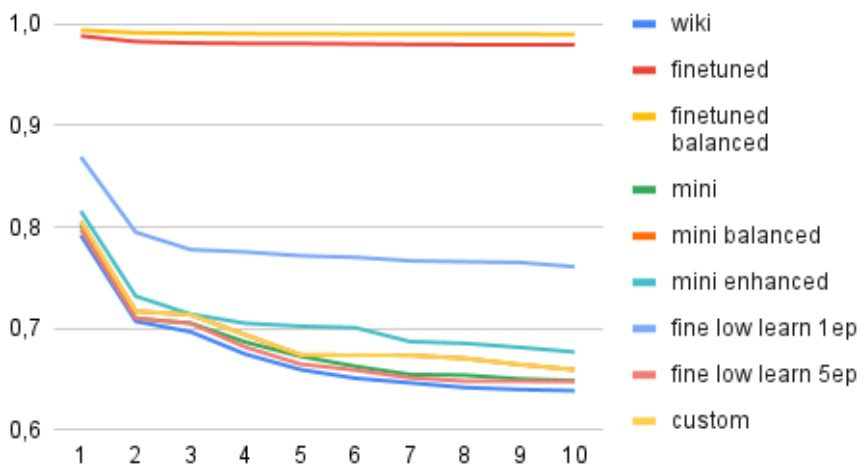
```
HOME_DIR = Path(".")
MODELS_DIR = HOME_DIR/"models"
DATA_DIR = HOME_DIR/"datasets"
DATASET_NAME = "irdb_corpus.csv"
TO_EMBED_FIELD_NAME = "text"
MODEL_NAME = "cc.it.300.bin"
MODEL_SHORT_NAME = "wiki"
FINE_TUNING_DATASET = MODELS_DIR / "IRDB_corpus_clean_event_descriptions"
FINE_TUNING_SHORT_NAME = "custom"
NEED_TO_GENERATE_FINETUNING_DATASET = True


def get_tokenized_list_from_file(file):
    return [simple_preprocess(row, min_len=0, max_len=100) for row in file]


if NEED_TO_GENERATE_FINETUNING_DATASET:
    df = pd.read_csv(DATA_DIR / DATASET_NAME)
    # Taking apart the sentences to embed and cleaning them
    df = df.get(TO_EMBED_FIELD_NAME).apply(lambda x: ' '.join(simple_preprocess(x)))

    # Saving the sentences to file so that Fasttext can pick-up them
    FINE_TUNING_DATASET = str(FINE_TUNING_DATASET) + "_" +
                          datetime.now().strftime("%Y%m%d%H%M%S") + ".bin"
    df.to_csv(FINE_TUNING_DATASET,
              index=False,
              sep=' ',
              header=False,
              quoting=csv.QUOTE_NONE,
              escapechar=" ")
    print("Saved new clean dataset: ", FINE_TUNING_DATASET)

# Loading model
print("Loading model...")
with open(MODELS_DIR/MODEL_NAME, 'rb') as fin:
    gensim_model = gensim.models.fasttext.load_facebook_model(fin)
print("Model loaded")
```

[6]https://commoncrawl.org/

```
with open(FINE_TUNING_DATASET) as f:
    file_lines = sum(1 for line in f)
    f.seek(0)
    print(f"Updating vocabulary with a {file_lines} lines corpus...")
    gensim_model.build_vocab(get_tokenized_list_from_file(f), update=True) # Update the vocabulary
    f.seek(0)
    print("Training model with new sentences...")
    sentences = get_tokenized_list_from_file(f)
    gensim_model.train(sentences,
                       total_examples=file_lines,
                       epochs=5,
                       start_alpha=gensim_model.alpha/10)

# Saving the model
output_file_path = str(MODELS_DIR/"model_") + MODEL_SHORT_NAME + "_finetuned_" +
                   FINE_TUNING_SHORT_NAME + "_" + str(gensim_model.epochs) + "epochs_" +
                   datetime.now().strftime("%Y%m%d%H%M%S") + ".bin"
gensim.models.fasttext.save_facebook_model(gensim_model, output_file_path)
print("Model saved: \n" + output_file_path)
```

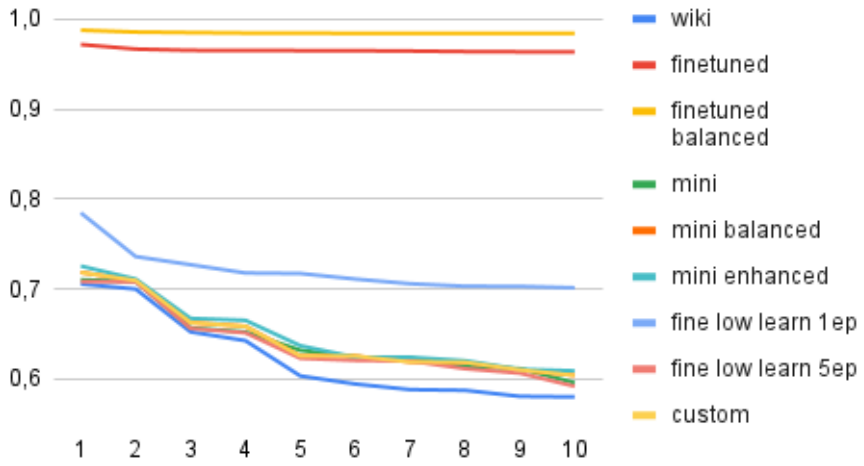
Listing 4.3: Creazione del modello *Wiki finetuned Custom*

Per poter confrontare i modelli sono state scelte alcune parole significative per il contesto in questione e sono stati confrontati i primi 10 risultati ottenuti richiedendo ai vari modelli le parole più simili a quella fornita:



custom		wiki		finetuned	
cadevano	0,8053	cadevano	0,7919	cadevano	0,9881
cade	0,7168	cadde	0,707	scendeva	0,9826
cadde	0,7135	cade	0,6967	scivolava	0,9811
scendeva	0,6937	scendeva	0,6746	cadendo	0,9807
cascava	0,674	cascava	0,6594	spostava	0,9806
perdeva	0,6737	perdeva	0,651	terminava	0,9803
cadendo	0,6735	finiva	0,6464	entrava	0,9798
scivolava	0,6705	ricadeva	0,6418	schiacciava	0,9796
spostava	0,6644	usciva	0,6398	trovava	0,9794
finiva	0,6593	scivolava	0,6386	precipitava	0,9794

Figura 4.4: Esempio di similarità tra word embedding utilizzando diversi modelli con la parola *cadeva*



custom		wiki		finetuned	
ematomi	0,7184	Ematoma	0,7056	ematomi	0,9715
Ematoma	0,7095	ematomi	0,6999	cefaloematoma	0,9664
edema	0,6627	edema	0,6524	contusivo	0,965
cefaloematoma	0,6586	cefaloematoma	0,6426	emorragico	0,965
contusivo	0,6256	ecchimosi	0,6035	aneurismatico	0,9647
sanguinamento	0,6256	contusione	0,5945	ascesso	0,9646
contusione	0,619	sanguinamento	0,5883	necrotica	0,9643
ecchimosi	0,6187	contusivo	0,5875	traumatismo	0,9637
ascesso	0,6103	tumefazione	0,5808	trauma	0,9634
tumefazioni	0,6041	angioma	0,5802	flogistico	0,9634

Figura 4.5: Esempio di similarità tra word embedding utilizzando diversi modelli con la parola *ematoma*

Dai grafici e dalle tabelle si nota immediatamente come le prestazioni di *custom* e *wiki* siano molto simili, infatti, se il primo si avvale della propria conoscenza del dominio, il secondo può contare sulla vastità di informazioni immagazzinate sulla lingua in cui è espresso il dominio. "Unendo" questi modelli per formare un modello *fine tuned*, i risultati fanno un notevole salto di qualità, decretando quindi il modello che verrà utilizzato.

4.2.2 Sentence embedding

Una volta scelto il modello per i word embedding, occorre realizzare i modelli di sentence embedding. Per realizzarli è stata utilizzata una libreria basata su Gensim chiamata *FSE*[7] (Fast Sentence Embeddings). Il motivo che ha portato all'utilizzo di questa libreria è che, sebbene Gensim fornisca tutti gli strumenti necessari al calcolo degli embedding (e infatti è alla base di FSE), non fornisce alcuno strumento "pronto all'uso" per la costruzione di sentence embedding. Al contrario, FSE fornisce classi come *Average* e *SIF* per il calcolo dei rispettivi embedding.

```
HOME_DIR = Path(".")
MODELS_DIR = HOME_DIR/"models"
CLEAN_EVENT_DESCRIPTIONS_NAME =
                        "IRDB_corpus_clean_event_descriptions_20220126134511.txt"

MODEL_NAME = "model_wiki_finetuned_custom.bin"
EMBEDDING_NAME = "wiki"

# Loading model
print("Loading Gensim word model...")
with open(MODELS_DIR / MODEL_NAME, 'rb') as fin:
    gensim_model = gensim.models.fasttext.load_facebook_model(fin)

# Prepare corpus for trainings
print("Preparing data...")
corpus = pd.read_csv(MODELS_DIR/CLEAN_EVENT_DESCRIPTIONS_NAME, header=None)
corpus_splitted_indexed = []
for i in range(len(corpus)):
    corpus_splitted_indexed.append(corpus.iloc[i, 0]. split ())
corpus_splitted_indexed = IndexedList(corpus_splitted_indexed)
```

[7] https://github.com/oborchers/Fast_Sentence_Embeddings

```
# Performing Average
print("Training Gensim Avg sentences model...")
average_model = Average(gensim_model, workers=16)
average_model.train(corpus_splitted_indexed)
average_model.save(str(MODELS_DIR/f"{EMBEDDING_NAME}_avg_model.bin"))
del average_model

# Performing IDF
print("Training Gensim IDF sentences model...")
idf_model = Idf(gensim_model, workers=16)
idf_model.train(corpus_splitted_indexed)
idf_model.save(str(MODELS_DIR / f"{EMBEDDING_NAME}_idf_model.bin"))

# Performing SIF
print("Training Gensim SIF sentences model...")
sif_model = SIF(gensim_model, workers=16)
sif_model.train(corpus_splitted_indexed)
sif_model.save(str(MODELS_DIR/f"{EMBEDDING_NAME}_sif_model.bin"))
del sif_model
```

Listing 4.4: Creazione dei modelli *Avg*, *IDF* e *SIF*

In realtà, FSE non ha una classe IDF, ragion per cui durante lo sviluppo dei modelli è stata realizzata sulla falsariga della classe SIF:

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# Author: Alessandro Muzzi <alessandromuzzi17@gmail.com>
# Copyright (C) 2021 Alessandro Muzzi
from typing import List

import numpy as np
from math import log
from fse.models.average import Average

from gensim.models.keyedvectors import KeyedVectors

from numpy import ndarray, float32 as REAL

import logging

logger = logging.getLogger(__name__)


class Idf(Average):
```

```
def __init__(self, model: KeyedVectors, sv_mapfile_path: str = None,
             wv_mapfile_path: str = None, workers: int = 1, lang_freq: str = None):
    """ Inverse document frequency (Idf)
        Because the term "the" is so common, term frequency will tend to incorrectly emphasize
        documents which happen to use the word "the" more frequently, without giving enough
        weight to the more meaningful terms "brown" and "cow". The term "the" is not a good
        keyword to distinguish relevant and non-relevant documents and terms, unlike the
        less-common words "brown" and "cow". Hence, an inverse document frequency factor is
        incorporated which diminishes the weight of terms that occur very frequently in the
        document set and increases the weight of terms that occur rarely. Karen Spärck Jones
        (1972) conceived a statistical interpretation of term-specificity called Inverse
        Document Frequency (idf), which became a cornerstone of term weighting:
            The specificity of a term can be quantified as an inverse function of the number
            of documents in which it occurs.

    Parameters
    ----------
    model : :class:`~gensim.models.keyedvectors.BaseKeyedVectors` or
            :class:`~gensim.models.base_any2vec.BaseWordEmbeddingsModel`
        This object essentially contains the mapping between words and embeddings.
        To compute the sentence embeddings the wv.vocab and wv.vector elements are required.
    sv_mapfile_path : str, optional
        Optional path to store the sentence-vectors in for very large datasets.
        Used for memmap.
    wv_mapfile_path : str, optional
        Optional path to store the word-vectors in for very large datasets. Used for memmap.
        Use sv_mapfile_path and wv_mapfile_path to train disk-to-disk without needing much ram.
    workers : int, optional
        Number of working threads, used for multithreading. For most tasks
        (few words in a sentence) a value of 1 should be more than enough.
    lang_freq : str, optional
        Some pre-trained embeddings, i.e. "GoogleNews-vectors-negative300.bin", do not contain
        information about the frequency of a word. As the frequency is required for estimating
        the word weights, we induce frequencies into the wv.vocab.count based on
        :class:`~wordfreq`. If no frequency information is available, you can choose the
        language to estimate the frequency. See https://github.com/LuminosoInsight/wordfreq
    """

    self.vocab = {}

    super(Idf, self).__init__(
        model=model, sv_mapfile_path=sv_mapfile_path, wv_mapfile_path=wv_mapfile_path,
        workers=workers, lang_freq=lang_freq)

def _pre_train_calls(self, **kwargs):
    """Function calls to perform before training """
    self._compute_idf_weights(kwargs)
```

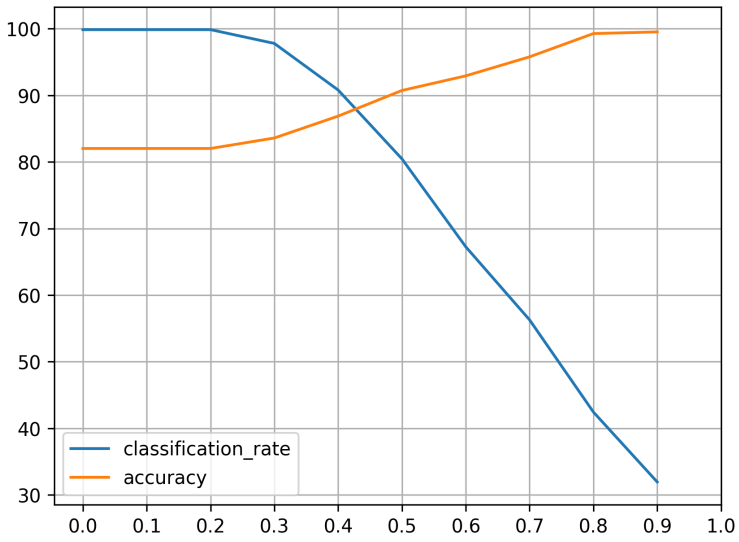
```
def _check_parameter_sanity(self):
    """ Check the sanity of all paramters """
    if not all(self.word_weights >= 0.):
        raise ValueError("For Idf, all word weights must be 0 <= w_weight <= 1")

def _compute_idf_weights(self, statistics):
    """ Computes the Idf weights for all words in the vocabulary """
    logger.info(f"pre-computing Idf weights for {len(self.wv)} words")

    words = self.wv.key_to_index
    ret = []
    if len(words) == 0:
        return np.zeros(self.wv.get_dimension())
    for word in words:
        count = self.vocab.get(word, 0)
        if count == 0:
            idf_w = 1
        else:
            idf_w = log(statistics['total_sentences'] / count, 10)

        ret.append(idf_w)

    self.word_weights = np.array(ret).astype(REAL)

def train(self, sentences: List[tuple] = None, update: bool = False, queue_factor: int = 2,
          report_delay: int = 5) -> [int, int]:
    """ Perform word count before start training to have the counts in _compute_idf_weights()
        called by _pre_train_calls() """

    for sentence, idx in sentences:
        sent = set(sentence)
        for word in sent:
            self.vocab[word] = self.vocab.get(word, 0) + 1

    return super().train(sentences, update, queue_factor, report_delay)
```

Listing 4.5: Classe Idf per FSE

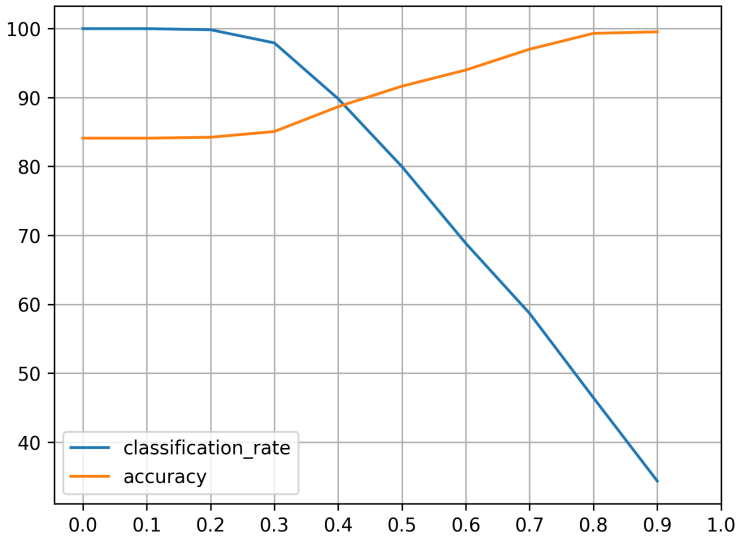
La classe Idf è stata sviluppata in modo da integrarsi senza problemi con le altre classi della libreria FSE e fa parte, assieme ad un porting di FSE per Gensim >= 4 (anch'esso realizzato in fase di sviluppo), di una pull request[8] su GitHub per annetterla alla libreria stessa. Ora che i modelli di sentence embedding sono pronti, occorre valutare la loro capacità di

[8] https://github.com/oborchers/Fast_Sentence_Embeddings/pull/65

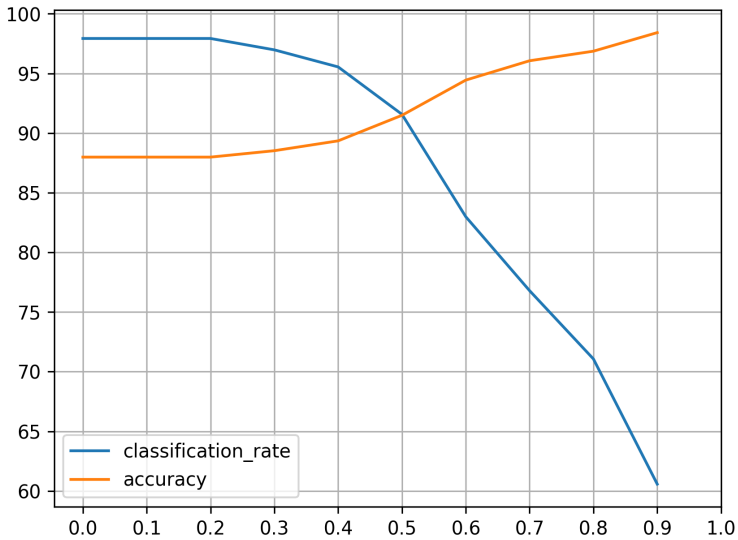
ottenere risultati soddisfacenti. Non potendo utilizzare tecniche di confronto basate sul ranking dei documenti esposte in Sezione 2.5.4 a causa della mancanza di test set di rilevanza dei documenti per validare i risultati, era necessario trovare uno strumento che fosse il più vicino possibile come funzionamento al motore di ricerca semantico. È stato quindi deciso di utilizzare un classificatore KNN, in quanto in grado di valutare l'efficacia semantica dei modelli realizzando cluster basati sulla loro vicinanza spaziale, utilizzando quindi di fatto la cosine similarity. In questo modo è possibile osservare la rappresentatività dei modelli ed utilizzare i risultati di questo esperimento come proxy per valutare la qualità della ricerca semantica, avendo a disposizione valori tangibili e confrontabili. Per rendere il task affidabile è stato realizzato manualmente un dataset contenente 630 elementi distribuiti su 9 classi, non necessariamente corrispondenti a quelle dell'IRDB.



(1) Avg



(2) IDF



(3) SIF

Figura 4.6: Test di classificazione con KNN per Avg (1), IDF (2) e SIF (3) con il modello Wiki finetuned IRDB

In figura 4.6 sono mostrati i risultati ottenuti dal KNN utilizzando le tecniche discusse in precedenza, al variare della soglia di confidenza applicata alla

classificazione. Il *classification rate* mostra infatti la percentuale di documenti classificati, che scende all'aumentare della soglia minima di confidenza richiesta per accettare la classificazione di un documento. Notiamo come il modello migliore risulti essere SIF, che con una soglia di confidenza minima del 50% (*0.5*) raggiunge il 91,57% di classification rate con un'accuratezza del 91,5%. Tuttavia, nonostante IDF performi leggermente peggio di SIF in questo task, occorre ricordare che l'obiettivo di questi modelli è quello di funzionare bene in task di calcolo di similarità. Questo esperimento di classificazione è stato realizzato per avere una metrica tangibile delle performance del modello. È per questo motivo che è stato svolto anche un sondaggio su un campione di utilizzatori del servizio chiedendo loro di indicare il modello che preferissero basandosi sui risultati ottenuti effettuando ricerche. Il modello scelto, tenendo conto anche del risultato del sondaggio, è il modello IDF.

Capitolo 5

Conclusioni

L'obiettivo di questa tesi era quello di realizzare un motore di ricerca semantico per segnalazioni in ambito sanitario, ovvero un motore di ricerca specializzato nell'ambito dell'*incident reporting* che fosse in grado di fornire non solo risultati che contengono le parole cercate, ma anche segnalazioni inerenti per tipo di incidente, luogo o dinamiche affini. Dai vari test effettuati, sia statistici che empirici, risulta che il motore di ricerca soddisfa pienamente i requisiti progettuali, riuscendo a fornire risultati inerenti a quanto richiesto con un buon grado di affidabilità. Dai 2 esempi in Figura 5.1 si può infatti osservare come i risultati siano inerenti al testo ricercato nonostante capiti che nessuna segnalazione in indice contenga la (o le) parola cercata, come nel secondo caso.

Scivola in bagno battendo la testa

Search

Recandosi in bagno scivola battendo la testa. vista dal mdg.	0.92
La paziente scivola a terra battendo la testa	0.91
La paziente cade mentre va in bagno, perdendo equilibrio, sbattendo la testa sulla sedia.	0.89
La paziente scendendo dal letto per andare in bagno perde una ciabatta e scivola a terra battendo bacino e nuca	0.88
Scivolata in bagno sbattendo la testa.	0.88
Il paziente si reca in bagno autonomamente e cade battendo la testa	0.88
Scivola in corridoio sbattendo la testa	0.87
La paziente che si trovava in bagno è scivolata a terra su una pozza d'acqua, battendo la testa contro il lavandino	0.87
Cade in camera battendo la testa a terra	0.87
Mentre il paziente esce dal bagno inciampa tra la porta d'ingresso e quella del bagno cadendo per terra e sbattendo la testa	0.87

molari

Search

Avulsione di protesi dentaria emiarcata sup.già precaria	0.47
Avulsione di 2 incisivi	0.47
Avulsione di due denti durante l'intubazione	0.46
Avulsione dentaria durante intubazione orotracheale (2 incisivo sup. sx)	0.44
Dimenticata protesi dentaria mobile in bocca	0.44
Durante l'intubazione oro tracheale si e' verificata l'avulsione accidentale di due denti incisivi , parzialmente instabili,dell'arcata superiore di sx.	0.43
Dimenticanza protesi dentaria	0.42
Durante ventilazione si evince lussazione completa incisivi inferiori	0.42
Perdita di protesi dentaria	0.42
Paziente con protesi dentaria in situ	0.41

Figura 5.1: Ricerche di esempio

5.1 Sviluppi futuri

Siccome l'obiettivo intrinseco di un motore di ricerca è quello di fornire per primi i risultati più pertinenti, dal punto di vista tecnico si tratta di ottimizzazione di un problema di ranking. Pertanto, tra gli sviluppi futuri già in programma vi è l'utilizzo dei dati raccolti dagli utenti attraverso un sistema di logging, ovvero utilizzando il feedback implicito (discusso in Sezione 2.5.4.2), per costruire un dataset di rilevanza volto a testare, validare e affinare la qualità del modello. In particolare, si potrebbe pensare ad un algoritmo di re-ranking dei risultati di ricerca, approccio che ha già dimostrato notevoli risultati[11]; questo algoritmo potrebbe essere addestrato utilizzando delle coppie (q, r) dove q è la query di ricerca e r il risultato selezionato dall'utente. Eventualmente sarebbe possibile espandere la coppia (q, r) in (q, r, s) aggiungendo il parametro s rappresentante lo score della risposta che potrebbe essere ottenuto raggruppando query simili con il motore di correlazione[1] e contando il numero di volte in cui quel determinato risultato è stato selezionato. Occorre però tenere a mente che il numero di query disponibili, essendo provenienti da un feedback implicito, dipende dall'utilizzo del sistema di ricerca che, sebbene sia esteso a tutta l'Emilia Romagna, verrà utilizzato solo dal personale sanitario, risultando quindi in un quantitativo stimato di $10^4 - 10^5$ query in un anno, probabilmente troppo poche per ottenere dei miglioramenti apprezzabili con le tecniche appena citate. Un modo per risolvere questo problema potrebbe consistere nel riutilizzare le query immesse dagli utenti per generare un ulteriore dataset da integrare con il precedente. Per farlo si potrebbero effettuare nuove ricerche con, ad esempio, un algoritmo come BM25[13] per poi utilizzare i risultati per creare triplette del tipo (q, p, n) dove q è la query di ricerca, p un esempio positivo, ovvero uno dei primi risultati, e n un esempio negativo, come uno degli ultimi risultati o un elemento non restituito dalla ricerca. In questo modo potremmo addestrare con un approccio debolmente supervisionato[12] una rete neurale,

[1] Componente già realizzato per SegnalER e riadattabile per questo tipo di task

utilizzando una *triplet loss* come funzione obiettivo, per implementare un meccanismo di re-ranking da applicare ai risultati dell'attuale motore di ricerca semantico.

Bibliografia

[1] Piotr Bojanowski, Edouard Grave, Armand Joulin, Tomas Mikolov, *Enriching Word Vectors with Subword Information*, Facebook AI Research, 2017.
https://arxiv.org/abs/1607.04606

[2] Arora, S., Liang, Y., & Ma, T. (2019). *A simple but tough-to-beat baseline for sentence embeddings*, paper presented at 5th International Conference on Learning Representations, ICLR 2017, Toulon, France. https://openreview.net/pdf?id=SyK00v5xx

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. *Attention Is All You Need*, 2017. https://arxiv.org/abs/1706.03762

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. *Bert: Pre-training of deep bidirectional transformers for language understanding*, Google AI Language, 2018. https://arxiv.org/abs/1810.04805

[5] Nils Reimers, Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, Ubiquitous Knowledge Processing Lab (UKP-TUDA), Technische Universitat Darmstadt, 2019. https://arxiv.org/abs/1908.10084

[6] Enrico Santus, Hongmin Wang, Emmanuele Chersoni, Yue Zhang. *A Rank-Based Similarity Metric for Word Embeddings*, Ubiquitous

Knowledge Processing Lab (UKP-TUDA), 2018. https://arxiv.org/abs/1805.01923

[7] Lowell K. Milliken, Sirisha K. Motomarry, Anagha Kulkarni. *ARtPM: Article Retrieval for Precision Medicine*, Department of Computer Science, San Francisco State University, San Francisco 94132, USA, 2019. https://doi.org/10.1016/j.jbi.2019.103224

[8] Jamin Shin Andrea Madotto Pascale Fung. *Interpreting Word Embeddings with Eigenvector Analysis*, CAiRE, The Hong Kong University of Science and Technology, 2018. https://openreview.net/pdf?id=rJfJiR5ooX

[9] Luyu Gao, Zhuyun Dai, Tongfei Chen, Zhen Fan, Benjamin Van Durme, Jamie Callan. *Complementing Lexical Retrieval with Semantic Residual Embedding*, Carnegie Mellon University, Johns Hopkins University, 2021. https://arxiv.org/abs/2004.13969

[10] Yukun Zhu, Ryan Kiros, Richard Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, Sanja Fidler. *Aligning Books and Movies: Towards Story-like Visual Explanations by Watching Movies and Reading Books*, University of Toronto & Massachusetts Institute of Technology, 2015. https://arxiv.org/abs/1506.06724

[11] Rodrigo Nogueira, Kyunghyun Cho. *Passage Re-ranking with BERT*, New York University, 2019. https://arxiv.org/abs/1901.04085

[12] Mostafa Dehghani, Hamed Zamani, Aliaksei Severyn, Jaap Kamps, W. Bruce Croft. *Neural Ranking Models with Weak Supervision*, University of Amsterdam, University of Massachuse's Amherst, Google Research, 2017. https://arxiv.org/abs/1704.08803

[13] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. *Okapi at TREC-3*, City University, Northampton Square, London, 1994. https://www.researchgate.net/publication/221037764_Okapi_at_TREC-3

[14] Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze. *Introduction to Information Retrieval*, Cambridge University Press, 2008.
https://nlp.stanford.edu/IR-book/html/htmledition/pseudo-relevance-feedback-1.html

[15] Alice Zheng, Amanda Casari. *Evaluating Machine Learning Models*, Sebastopol, CA: O'Reilly Media, 2015.
https://www.oreilly.com/library/view/feature-engineering-for/9781491953235/ch04.html

[16] Alexander Budanitsky, Graeme Hirst. *Evaluating wordnet-based measures of semantic distance. Comutational Linguist*, University of Toronto, 2006. https://aclanthology.org/J06-1003.pdf

[17] Leong, Chee Wee and Mihalcea, Rada. *Measuring the semantic relatedness between words and images*, Carnegie Mellon University, Johns Hopkins University, 2021. https://aclanthology.org/W11-0120

[18] Kashyap Kathrani. *All about Embeddings*, 2020. https://medium.com/@kashyapkathrani/all-about-embeddings-829c8ff0bf5b

[19] Renu Khandelwal. *Simple Explanation of Transformers in NLP*, 2020.
https://towardsdatascience.com/simple-explanation-of-transformers-in-nlp-da1adfc5d64f

[20] Intellica AI. *Comparison of different Word Embeddings on Text Similarity — A use case in NLP*, 2019.
https://intellica-ai.medium.com/comparison-of-different-word-embeddings-on-text-similarity-a-use-case-in-nlp-e83e08469c1c

[21] Ryan Brideau. *Precision@k: The Overlooked Metric for Fraud and Lead Scoring Models*, 2021.

https://towardsdatascience.com/precision-k-the-overlooked-metric-for-fraud-and-lead-scoring-models-fabad2893c01

[22] Renu Khandelwal. *Word Embeddings for NLP*, 2019.
https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4

[23] Sanjana Reddy. *GloVe and fastText — Two Popular Word Vector Models in NLP*, 2019.
https://blogs.sap.com/2019/07/03/glove-and-fasttext-two-popular-word-vector-models-in-nlp/

[24] Côme Cothenet. *Short technical information about Word2Vec, GloVe and Fasttext*, 2020.
https://towardsdatascience.com/short-technical-information-about-word2vec-glove-and-fasttext-d38e4f529ca8

[25] Ajit Rajasekharan. *A review of BERT based models*, 2019.
https://towardsdatascience.com/a-review-of-bert-based-models-4ffdc0f15d58

[26] *Elasticsearch*.
https://www.elastic.co/